PREMIUM EDITION · 2026

The Prompt

The Complete System for Building, Testing, and Deploying Production AI Prompts

8 Template Schemas · 7 Persuasion Principles · 8 Development Patterns · 100+ Battle-Tested Templates

30	100+	434	18mo
CHAPTERS	TEMPLATES	VIDEOS SYNTHESIZED	PRODUCTION USE

Synthesized from Anthropic's official documentation, 434 YouTube videos, academic research, and 18 months of production use across real AI pipelines.

CONTENTS

Table of Contents

PART III

The System Layer - Your Unique Differentiator

PART IV

The Psychology of Prompting

PART V

Agentic Patterns - Multi-Agent Systems

PARTS VI–VIII

Patterns, Anti-Patterns & Applied Domains

PART I

Foundations
The Mental Models

Before techniques come mental models. The practitioners who get 10x results from AI don't know more tricks - they think about the problem differently. These three chapters install the operating system for everything that follows.

CHAPTER 01

The Mental Model Shift

> *Most people treat prompts like search queries. Professionals treat them like source code. This single shift in perspective separates practitioners who get consistent, production-grade results from those who remain permanently frustrated.*

From Prompts to Systems

The average person writing AI prompts operates in one-shot mode: type something, hope for something useful, tweak and retry. This approach works occasionally - the same way randomly hitting piano keys sometimes produces a pleasant sound. But it doesn't scale, can't be versioned, and produces wildly inconsistent results across team members or deployments.

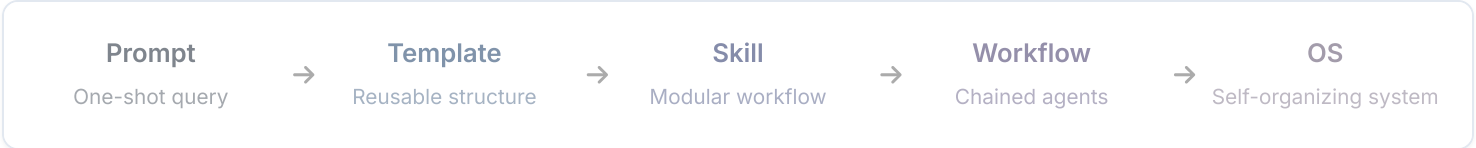
Professional prompt engineering is about building *systems*, not crafting one-off queries. A system has structure, state, composable components, and verifiable behavior. When something breaks, you know where to look. When something works, you can replicate it. When requirements change, you can modify a specific component without rewriting everything.

The insight that changes everything: **a prompt is not a message to an AI. It is a specification for a deterministic (enough) computation.** When you write a prompt, you're programming - just in natural language instead of code. Every principle that makes code maintainable applies directly to prompts: single responsibility, separation of concerns, explicit over implicit, testability, and version control.

> 💡 **PARADIGM SHIFT**
>
> Stop asking "how do I phrase this better?" Start asking "how do I architect this system to produce reliable output?" The first question is about words. The second is about engineering.

The Spectrum: Prompt → OS

There's a full maturity spectrum from single prompts to complete AI operating systems. Understanding where you are - and where you want to be - determines which techniques to invest in.



A **Prompt** is a single instruction to an AI. A **Template** is a prompt with variables - reusable, parameterizable, slightly more consistent. A **Skill** is a modular, self-contained workflow packaged with metadata, examples, and documentation - callable by other agents or humans. A **Workflow** chains skills together, with outputs from one becoming inputs to the next. An **Operating System** is a complete environment: an orchestration layer, memory system, tool registry, quality assurance loop, and self-improvement mechanism. This book will take you from wherever you are now to building operating systems.

Why 90% of Prompt Engineering Advice Is Wrong

The dominant paradigm in prompt engineering YouTube videos, Twitter threads, and blog posts treats prompts as *static text artifacts*. The advice centers on magic phrases: "act as an expert," "think step by step," "respond only in JSON." These tips are not wrong - they work sometimes. But they're optimizing for a one-shot interaction, not a production system.

Static prompts decay. As models update, context windows grow, and your system evolves around them, static prompts stop working and you don't know why. They can't be tested systematically. They can't be composed. They can't be improved by data. The entire "prompt hacks" cottage industry is built on the wrong abstraction.

✕ STATIC PROMPT MINDSET

```
// Stored as a string in a config file
const PROMPT = "You are a helpful assistant that
writes great code. Think step by step.
Always be thorough and accurate.
Please help me with the following:"
```

✓ SYSTEM MINDSET

```
// skill.md  -  versioned, testable, composable
---
name: code-generator
version: 2.3.1
model: claude-sonnet
tests: tests/code-generator.yaml
---
IDENTITY: Expert software engineer
CONSTRAINTS: [testable, documented]
VALIDATION: run tests, lint, type-check
OUTPUT: code + explanation + tests
```

The "Prompt = Code" Principle

Treat every production prompt with the same discipline you'd apply to source code. This means four things in practice:

- **Version control.** Every prompt lives in a file, tracked in git. Changes are committed with messages explaining what changed and why. You can diff two versions, roll back, and see the history of your thinking.
- **Tests.** Every prompt has a test suite - a set of inputs with expected output characteristics. Tests run on every change and before any deployment. Regression detection is automatic.
- **Composability.** Prompts are designed as modules. They accept well-defined inputs, produce well-defined outputs, and can be combined without knowing each other's internals.
- **Documentation.** Every prompt documents its purpose, inputs, outputs, examples, and known failure modes - the equivalent of a docstring and README.

> *"The single biggest difference between amateur and professional prompt engineering is version control. The amateur rewrites their prompt and hopes it's better. The professional tracks every change and knows exactly what made it better."*
>
> **- Synthesis from 434 analyzed videos**

CHAPTER 02

How LLMs Actually Process Your Input

> *You don't need a PhD in machine learning to use this knowledge. But understanding the basics of how language models read your prompts will explain every technique in this book - and help you invent new ones.*

Tokens, Attention, and Context Windows

Language models don't read text. They read tokens - chunks of characters that sit somewhere between letters and words. The word "tokenization" is two tokens. "Prompt" is one. "GPT-4" might be three. This matters because models have context windows measured in tokens, not words (roughly, 1 token ≈ 0.75 words in English).

The attention mechanism is what allows a model to relate any token to any other token in its context. When you write "the previous code had a bug in the authentication function - fix it," the model attends to both "authentication function" and "fix it" simultaneously, relating them across distance. This is fundamentally different from how earlier NLP systems worked. The implication: position matters less than you think, but structure matters more.

Context windows have grown dramatically: from 4K tokens (GPT-3) to 200K (Claude 3) to 1M+ (Gemini 1.5). This doesn't mean you should fill them. Large contexts introduce latency, cost, and - critically - diluted attention. Important information in a sea of irrelevant text gets less attention. The model still reads it all, but the signal-to-noise ratio drops.

> ⚠️ **THE CONTEXT WINDOW MISCONCEPTION**
>
> "I have 200K tokens, so I'll just paste everything." This is the most common mistake in production AI systems. More context = more noise = worse performance on the actual task. The goal is precision, not volume.

The "Smart Zone" Concept

Based on empirical testing across multiple models and task types, AI systems tend to perform best when context utilization is between 30–70% of the context window. Below 30%, the model may lack sufficient information. Above 70%, attention dilution begins to degrade quality - especially for tasks requiring reasoning over the full context.

<30%	30–70%	>70%
Under-informed - model lacks context	Smart Zone - optimal performance	Attention dilution - quality drops

In practice, this means: summarize rather than dump, chunk long documents rather than passing full text, and use retrieval systems to pull only relevant context rather than including everything. The discipline of keeping context in the smart zone is one of the highest-leverage skills in production prompt engineering.

Why Structure Matters More Than Length

A 50-token prompt with clear structure will consistently outperform a 500-token prompt that's a wall of text. The attention mechanism doesn't care about word count - it cares about the patterns and relationships between tokens. When you add XML tags, headers, or numbered lists, you're creating semantic boundaries that the model's attention mechanism can use to segregate concepts.

This is why a prompt like `<task>Summarize this</task><text>{content}</text>` outperforms "Here is some text I need you to summarize: {content}" - not because of magic, but because the tag boundaries create unambiguous semantic regions that the attention mechanism can cleanly separate.

Why XML Tags Work

Claude in particular was trained extensively on documents containing XML-like markup. As a result, Claude's attention patterns are strongly conditioned to treat XML tags as semantic boundaries - similar to how a human reader treats headings and section breaks. When you wrap your instructions in `<instructions>` tags and your context in `<context>` tags, you're exploiting a feature of the model's training, not just adding decoration.

The effect is measurable: in Anthropic's own research, structured prompts with XML tags produce more consistent outputs, better adherence to format requirements, and lower hallucination rates on knowledge-grounded tasks. Other models (GPT-4, Gemini) respond well to Markdown headers for similar reasons - their training data included heavily structured Markdown documents.

Temperature, Top-p, and When They Matter

Temperature controls the "randomness" of sampling. At temperature 0, the model always picks the highest-probability next token - deterministic and consistent, but sometimes dull. At temperature 1, it samples from the full probability distribution - more creative, but less predictable. In practice:

TEMPERATURE	BEST FOR	AVOID FOR
`0.0`	Code generation, data extraction, structured output	Creative writing, brainstorming
`0.3`	Technical writing, summarization, analysis	Open-ended ideation
`0.7`	General conversation, drafting, balanced tasks	Tasks requiring exact reproducibility
`1.0`	Creative writing, poetry, brainstorming	Production pipelines, data tasks

Top-p (nucleus sampling) is an alternative to temperature. Rather than scaling the whole distribution, it samples from only the top tokens whose cumulative probability exceeds p. Most practitioners set top-p to 1.0 and adjust temperature instead - the two controls are partially redundant and adjusting both simultaneously adds confusion. **Pick one: temperature.**

> 💡 **PRODUCTION DEFAULT**
>
> For production AI pipelines handling tasks like data extraction, code generation, or structured output: set temperature to 0 and forget it. Reserve higher temperatures for explicitly creative tasks where variation is a feature, not a bug.

CHAPTER 03

The Core Four Framework

> *Every AI interaction - from a one-line query to a 50-agent pipeline - optimizes the same four levers. Master these four and you have a universal mental model for diagnosing problems and finding improvements in any AI system.*

This framework, synthesized from IndyDevDan's foundational work and extended through production deployments, provides a complete decomposition of what you can actually control in an AI interaction. When a pipeline isn't performing, it's always a problem with one or more of these four levers. When you improve a system, you're always improving one or more of these four.

LEVER	WHAT IT CONTROLS	OPTIMIZATION TARGET	COMMON MISTAKE
Context	What the agent knows - data, history, documents, conversation	Minimize to essential; use progressive disclosure	Dumping everything "just in case"
Model	Which LLM processes the request - capability and cost	Match model to task complexity	Using Opus for tasks Haiku handles fine
Prompt	How the agent behaves - instructions, role, constraints, format	Structure > length; consistency > cleverness	Writing essays when bullet points work
Tools	What the agent can do - external capabilities, APIs, functions	Load on demand; fewer focused tools > many generic	Giving agents 50 tools when they need 3

Lever 1: Context

Context is everything the model can attend to: your system prompt, conversation history, retrieved documents, tool outputs, and any other information in the context window. It's the most mismanaged lever. The natural instinct is to provide more context - "the more it knows, the better it'll do." This intuition is wrong for two reasons.

First, attention dilution. In a context window with 50,000 tokens, a critical constraint buried in token 23,000 gets less weight than one in token 100. Second, cognitive load. Models, like humans, perform worse when overwhelmed with information irrelevant to the current task. A model asked to fix a bug in a 3,000-line file will underperform the same model asked to fix the same bug in a 200-line excerpt containing only the relevant function.

The discipline: ruthlessly cut context to only what's needed for the current step. Use retrieval systems to pull relevant excerpts instead of dumping full documents. Use progressive disclosure - provide L0 (name) at first, escalate to L3 (full detail) only when needed. Think of context as RAM: it's finite, it's expensive, and loading it with junk slows everything down.

Lever 2: Model

Model selection is a cost-capability tradeoff that most practitioners get wrong in both directions. The most common mistake is using the most capable (and most expensive) model for everything. Anthropic's model family illustrates the principle:

MODEL TIER	USE WHEN	AVOID WHEN	COST SIGNAL
Haiku / Flash	Classification, extraction, simple formatting, routing decisions	Complex reasoning, creative synthesis, nuanced judgment	~10–20x cheaper than frontier
Sonnet / Pro	Most production tasks - code, analysis, writing, summarization	Trivial tasks that Haiku handles fine	Mid-range; production sweet spot
Opus / Ultra	Final synthesis, high-stakes decisions, complex multi-step reasoning	Any task that doesn't require maximum reasoning capability	Premium; reserve for synthesis

The pattern that works in production: Haiku for subtasks (routing, extraction, formatting), Sonnet for primary work (code generation, analysis), Opus for final judgment (quality review, synthesis, high-stakes decisions). A well-architected multi-agent system uses all three tiers, dramatically reducing cost while maintaining quality.

Lever 3: Prompt

The prompt is the interface to the model - the specification that tells it what to do, how to behave, and in what format to respond. It's the most written-about lever and the most over-engineered. The discipline here is the opposite of what most prompt engineers do: instead of trying to cover every edge case with longer and more elaborate instructions, write shorter and more structural prompts.

The key insight: **structure beats length**. A 200-token prompt with clear XML structure, an explicit role, three focused constraints, and one output format specification will outperform a 2,000-token prompt that's a rambling paragraph trying to anticipate every scenario. Models are better at following clear, bounded instructions than at extracting intent from verbose prose.

Lever 4: Tools

Tools transform models from knowledge repositories into capable agents. A model with a web search tool can answer questions about current events. A model with a code execution tool can verify its own outputs. A model with a database tool can retrieve and update real records.

The optimization principle: load tools on demand, not all at once. Providing an agent with 50 available tools increases the probability of tool misuse, confuses the model about which tool to use for a given task, and wastes context tokens with tool definitions. Instead, give each agent the minimum set of tools required for its specific function. A research agent gets web search and URL fetch. A code agent gets file read/write and shell execution. A review agent gets no tools - it only reads.

> ✅ **CORE FOUR DIAGNOSTIC**
>
> When an AI system isn't performing well, run through the four levers as a diagnostic checklist: Is context too large or too noisy? Is the model under-powered for the complexity? Is the prompt unclear or over-long? Do the tools provide what the agent actually needs? 90% of production issues are solved by fixing one of these four levers - not by finding a magic prompt phrase.

PART II

The Technique Hierarchy What Actually Works

Anthropic's research identifies a clear ordering of prompt engineering techniques by impact. We follow that hierarchy here - starting with the most powerful and working toward the more specialized. Most practitioners over-invest in advanced techniques while underinvesting in the basics. Read these chapters in order.

CHAPTER 04

Clarity and Directness

> *The highest-impact prompt engineering technique is the simplest: say exactly what you want. Not approximately. Not with hedges and qualifiers. Exactly. This chapter is about developing the discipline to be specific when vagueness is comfortable.*

The #1 Technique

If you could only apply one prompt engineering technique for the rest of your life, it would be this: state your requirement with the precision of a legal contract. Every ambiguous word in your prompt is a branching point where the model chooses - and often chooses differently than you intended. Eliminate ambiguous words and you eliminate unpredictable behavior.

Language models are trained to be helpful. When your prompt is vague, the model doesn't refuse - it guesses what you mean and generates the most probable continuation. That guess is often reasonable. It's rarely exactly what you wanted. Specificity doesn't just improve output quality; it reduces variance. The same specific prompt run 10 times will produce 10 similar outputs. The same vague prompt run 10 times will produce 10 very different outputs.

The Newspaper Test

Before submitting any prompt, apply this test: if a journalist read your prompt without knowing you, would they know exactly what output you expect? Could they describe your success criteria? If the answer is no, your prompt is not specific enough.

This test forces you to make implicit assumptions explicit. "Write a good blog post about AI" fails the newspaper test - "good" is undefined, "about AI" covers a universe. "Write a 600-word blog post for software engineers, covering three concrete ways Claude's extended thinking feature differs from standard inference, with at least one code example per point, published tone (not academic), no introduction paragraph" passes the newspaper test.

5 Bad vs Good Examples

× VAGUE

```
Write me a summary of this document.
```

✓ SPECIFIC

```
Write a 3-bullet executive summary of the
attached document. Each bullet: max 25 words.
Focus on financial implications only.
Audience: CFO with no technical background.
```

✕ VAGUE

```
Help me improve this code.
```

✓ SPECIFIC

```
Review this Python function for:
1. Time complexity (flag anything above O(n log n))
2. Missing error handling (None inputs, empty lists)
3. PEP 8 violations
Output: a diff with inline comments. No refactoring.
```

✕ VAGUE

```
Analyze this data and give me insights.
```

✓ SPECIFIC

```
Given this CSV of monthly sales data (2023):
1. Identify the 3 months with highest MoM growth
2. Calculate the correlation between ad spend
   and revenue (Pearson r)
3. Flag any months where returns exceeded 15%
   of gross sales
Format: JSON with keys: top_months, correlation, flags
```

✕ VAGUE

```
Make this email better.
```

✓ SPECIFIC

```
Rewrite this email to:
- Cut word count by 40% (current: ~200 words)
- Lead with the ask, not the background
- Use plain language (no jargon)
- End with a single, clear CTA
Preserve: all specific dates, names, dollar amounts.
Do not change: subject line.
```

✕ VAGUE

```
Translate this to Spanish.
```

✓ SPECIFIC

```
Translate this text to Mexican Spanish (es-MX).
Register: formal (usted, not tú).
Technical terms: keep in English with Spanish
  explanation in parentheses on first use.
Do not localize: brand names, product names,
  model numbers.
```

Specificity Patterns

Several patterns reliably increase specificity without making prompts excessively long:

- **Constrain the output format** - "JSON with keys X, Y, Z" beats "give me structured output"
- **Quantify requirements** - "3 bullet points, max 20 words each" beats "concise bullets"
- **Name the audience** - "for a non-technical CFO" beats "for a business audience"
- **Specify what to exclude** - "no introduction paragraph" beats "be direct"
- **Define ambiguous terms** - "comprehensive means: covers the 5 use cases listed below" beats "be comprehensive"

- **Use imperative voice** - "List three options" beats "Can you list some options?"

> ⚠️ **THE POLITENESS TRAP**
>
> Phrasing instructions as requests ("Could you please...") adds tokens without adding clarity. Models don't need politeness - they need precision. "Summarize in 3 bullets" is strictly better than "Could you possibly summarize this in maybe 3 bullets?" The former is a specification. The latter is a suggestion the model can deviate from.

CHAPTER 05

Few-Shot Learning & Example Design

> *Anthropic consistently ranks providing examples as the second most effective prompt engineering technique. The reason is fundamental: showing is almost always clearer than telling. A model that has seen three examples of what you want knows more than one that's read a paragraph describing it.*

Why Examples Are #2

Language models learn by predicting patterns in text. When you provide examples in a prompt, you're activating the same learning mechanism that trained the model - but locally, in context, for your specific task. The model generalizes from your examples to the new input, leveraging its deep pattern-matching capability in a directed way.

This is called *in-context learning*: no weights are updated, no fine-tuning happens, but the model adapts its behavior based on the examples it sees. The effect is substantial. Studies from Anthropic, OpenAI, and academic groups consistently find that 2–5 well-designed examples improve task performance by 15–40% over zero-shot approaches for complex or nuanced tasks.

How Many Examples

The research-backed sweet spot is **2–5 examples** for most tasks. Below 2, the pattern is underspecified - one example could be a coincidence. Above 5, returns diminish rapidly while context costs grow. There are exceptions:

- **Use 1 example** when the task is simple and the example is more for format than content
- **Use 5+ examples** for highly constrained tasks where edge cases need explicit demonstration, or when the task requires learning a very specific style or voice
- **Use 10+ examples** only when you're essentially doing prompt-based fine-tuning - teaching a complex classification system or a highly stylized writing pattern

Example Structure: Input → Reasoning → Output

The most powerful example structure includes not just input and output, but reasoning - a brief explanation of *why* the output looks the way it does. This structure uses chain-of-thought (covered in the next chapter) at the example level, teaching the model both the what and the why.

```
✓ GOOD

# Example structure: Input → Reasoning → Output

<example>
<input>
Customer message: "I've been waiting 3 weeks and nothing.
This is unacceptable. I want my money back."
</input>

<reasoning>
Customer is frustrated, not hostile. "3 weeks" is a specific
grievance. "I want my money back" is a clear ask. Response
should acknowledge the specific wait, validate frustration,
offer a concrete next step, not just an apology.
</reasoning>

<output>
I'm sorry it's taken 3 weeks  -  that's too long, and your
frustration is completely understandable. I've pulled up
your order and I'm escalating it to priority processing
right now. You'll receive a tracking update within 4 hours.
If you'd prefer a refund instead, I can process that
immediately. Which would you like?
</output>
</example>
```

Edge Case Examples

The most valuable examples are not representative cases - those the model handles fine without guidance. The most valuable examples are *edge cases*: the inputs that are ambiguous, unusual, or where your desired behavior might not be obvious. Edge case examples define the boundaries of acceptable behavior.

> ✅ **EXAMPLE**
>
> If you're building a sentiment classifier, don't show examples of clearly positive ("I love this!") and clearly negative ("This is terrible!") reviews. The model already handles those. Show examples of the hard cases: sarcasm ("Oh great, another delay"), mixed sentiment ("The food was amazing but the service was awful"), and domain-specific neutral language that looks negative to a general model ("The compound inhibited growth at 50µM concentration").

Common Mistakes in Example Design

- **Inconsistent format.** If some examples have reasoning traces and others don't, the model will sometimes include them and sometimes not. Be consistent.
- **Examples that contradict each other.** Two examples where the same input type gets different outputs will produce unpredictable behavior. Each example should reinforce the same underlying rule.
- **Too similar.** Five examples of the exact same type of input teach the model nothing about generalization. Vary your examples across the distribution of real inputs.
- **Missing the output format.** If your examples don't demonstrate the exact output format you want (JSON, Markdown, specific schema), the model will invent its own.
- **Using bad inputs.** Examples with typos, grammatical errors, or ambiguity in the input portion teach the model to tolerate ambiguous input. Use clean, representative inputs unless you're specifically teaching the model to handle messy data.

Template for Effective Examples

```
# Few-shot template  -  copy and fill for your task

<examples>

<example id="1">
<input>[Representative, clean input  -  typical case]</input>
<reasoning>[Why does output look the way it does?]</reasoning>
<output>[Exact format you want. Consistent across all examples.]</output>
</example>

<example id="2">
<input>[Edge case  -  ambiguous or unusual input]</input>
<reasoning>[Why this edge case is handled this specific way]</reasoning>
<output>[Shows boundary behavior clearly]</output>
</example>

<example id="3">
<input>[Harder case  -  requires more nuanced reasoning]</input>
<reasoning>[Walk through the decision explicitly]</reasoning>
<output>[Output that reflects the nuanced reasoning above]</output>
</example>

</examples>
```

<task>Now apply the same pattern to:</task>
<input>[Actual input]</input>

CHAPTER 06

Chain-of-Thought & Extended Thinking

> *Chain-of-thought prompting unlocks reasoning capability that's latent but inaccessible without the right trigger. When you ask a model to show its work, you're not just getting transparency - you're changing what computations are performed.*

Zero-Shot CoT

The simplest form: append "Think through this step by step" or "Let's reason through this carefully before answering" to your prompt. This single addition unlocks dramatically improved performance on multi-step reasoning tasks - arithmetic, logic puzzles, planning problems, and causal analysis.

Why does it work? When a model is forced to produce reasoning tokens before the answer, those intermediate tokens become context for the final answer. The answer is now conditioned on a chain of valid reasoning steps rather than being generated directly from the question. It's the equivalent of showing your work on a math exam: the act of writing out the steps reduces errors.

✕ DIRECT (ERROR-PRONE ON COMPLEX PROBLEMS)

```
A train travels 120 miles in 2 hours. It then
travels 80 miles in 90 minutes. What is the
average speed for the whole journey?

Answer: [model often errors]
```

✓ CHAIN-OF-THOUGHT

```
A train travels 120 miles in 2 hours. It then
travels 80 miles in 90 minutes. What is the
average speed for the whole journey?

Think step by step before answering.

Step 1: Total distance = 120 + 80 = 200 miles
```

```
Step 2: Total time = 2hr + 1.5hr = 3.5 hours
Step 3: Avg speed = 200/3.5 = 57.1 mph ✓
```

Few-Shot CoT

Provide examples that include the reasoning trace, not just input/output. This is more powerful than zero-shot CoT because you're teaching the model *how* to reason about your specific problem type, not just asking it to reason in general. The model learns your preferred reasoning style, depth, and structure.

```
<example>
<problem>Should we launch this feature given: 3 weeks of dev time,
$50K expected revenue uplift, 60% confidence in estimate,
competing priority is a security fix?</problem>

<reasoning>
1. Risk-adjusted value: $50K × 0.60 = $30K expected value
2. Security fix is non-negotiable  -  it's compliance, not optional
3. Can both be parallelized? Check team capacity
4. 3 weeks of dev time vs $30K EV → $10K/week, above our $7K threshold
5. Verdict: launch after security fix, or in parallel if capacity allows
</reasoning>

<decision>PROCEED  -  after security fix, or parallel if team allows</decision>
</example>
```

When CoT Helps vs Hurts

TASK TYPE	COT EFFECT	REASON
Multi-step arithmetic	✅ Strong improvement	Intermediate steps prevent compounding errors
Logical reasoning / puzzles	✅ Strong improvement	Forces explicit constraint tracking
Causal analysis	✅ Improvement	Chain forces cause→effect separation
Code debugging	✅ Improvement	Reasoning reveals assumptions

TASK TYPE	COT EFFECT	REASON
Simple classification	⚠️ Slight degradation	Overthinking simple patterns
Factual lookup	❌ Neutral/negative	The answer is direct - reasoning adds noise
Creative writing	❌ Degrades flow	Analytical frame disrupts creative generation

Extended Thinking in Claude

Claude's extended thinking feature takes CoT to an architectural level. Rather than reasoning in the response, Claude reasons in hidden "thinking blocks" - internal scratchpad tokens that are processed before the response is generated. The key difference: thinking blocks can explore dead ends, backtrack, and try multiple approaches without those exploratory tokens appearing in the final output.

Extended thinking is most valuable for: complex math and code, multi-step planning problems, tasks where exploring multiple approaches before committing is valuable, and situations where you want a clean response but need heavy-duty reasoning behind it. Enable it via the API's `thinking` parameter with a `budget_tokens` limit.

```
# Claude API  -  extended thinking
{
  "model": "claude-opus-4-5",
  "max_tokens": 16000,
  "thinking": {
    "type": "enabled",
    "budget_tokens": 10000  // Internal reasoning budget
  },
  "messages": [{
    "role": "user",
    "content": "Design the optimal database schema for a
    multi-tenant SaaS with row-level security,
    audit logging, and sub-100ms query performance
    at 10M rows per tenant."
  }]
}
```

Structured Thinking Patterns

For decision-making and analysis tasks, provide a thinking scaffold that structures the reasoning rather than leaving it freeform. This is particularly useful in agentic systems where reasoning needs to be auditable:

```
<thinking_scaffold>
Before responding, reason through these steps:

PROBLEM: What exactly is being asked? What are the constraints?

ANALYSIS: What information do I have? What's missing?
  What assumptions am I making?

OPTIONS: What are 2-3 distinct approaches?
  (Don't commit yet  -  explore first)

EVALUATION: For each option: pros, cons, risks, fit with constraints.

DECISION: Which option? Why? What are the failure modes?
</thinking_scaffold>

<task>{task}</task>
```

CHAPTER 07

XML Structure & Semantic Markup

> *XML tags are the most underrated prompt engineering technique. They transform flat text into structured documents with unambiguous semantic regions, and Claude in particular has deep conditioning to respect them.*

Why Claude Responds Exceptionally Well to XML

Claude's training data included vast amounts of XML-structured content: documentation, configuration files, structured data, and Anthropic's own internal formatting conventions. As a result, Claude's attention mechanism has strong priors around XML tag boundaries - it treats the content of `<instructions>` tags as instructions and the content of `<context>` tags as context, not because it was told to, but because that pattern is deeply encoded in its weights.

The practical implication: XML tags do semantic work that prose can't. Writing "Now here is the context you should use:" followed by the context is weaker than `<context>` ...context... `</context>`. The tag creates a hard boundary that the attention mechanism respects; the prose creates a soft boundary that it may or may not.

Core Tags

TAG	PURPOSE	BEST PRACTICE
`<instructions>`	Primary behavioral directives	Use for the "what to do" - one per prompt
`<context>`	Background knowledge, state	Everything the model needs to know but not act on directly
`<examples>`	Few-shot examples	Always nest individual examples in `<example>` sub-tags
`<task>`	The specific work item	Put last - model reads sequentially
`<output>`	Format specification	Explicit schema, not vague description
`<constraints>`	Hard rules and limits	Use for non-negotiable requirements

TAG	PURPOSE	BEST PRACTICE
`<thinking>`	Internal reasoning space	Ask model to reason here before responding

Nesting and Named Documents

XML tags compose naturally. For complex prompts with multiple documents, use named document tags to give each piece of content a clear identity:

```
<context>
  <document name="product-spec" version="2.1">
  [Product specification content]
  </document>

  <document name="previous-feedback">
  [User research findings]
  </document>

  <document name="technical-constraints">
  [Engineering constraints]
  </document>
</context>

<instructions>
Review the product-spec against the technical-constraints
and list any conflicts. Then cross-reference with
previous-feedback to identify the three highest-priority
conflicts to resolve first.
</instructions>
```

Notice how the instructions can reference documents by name. This is only possible because named document tags give each piece of content an unambiguous identity.

Complete XML Prompt Template

```
<system>
[Identity and role definition]
[Core behavioral constraints]
```

```
[Tone and communication style]
</system>

<context>
<document name="[name]">[Relevant background]</document>
<document name="[name]">[Additional context]</document>
</context>

<examples>
<example>
<input>[Example input]</input>
<output>[Example output]</output>
</example>
</examples>

<constraints>
- [Hard constraint 1]
- [Hard constraint 2]
</constraints>

<task>
[The specific request  -  goes last]
</task>

<output_format>
[Exact schema or format specification]
</output_format>
```

⚠ **WHEN NOT TO USE XML**

Simple one-line prompts do not benefit from XML tags. "Translate this to French: {text}" is correct - no tags needed. Tags add overhead and visual noise that's not justified for trivial prompts. Use XML when you have multiple distinct semantic components that need clear separation: instructions + context + examples + task. If you only have one component, skip the tags.

CHAPTER 08

Role Assignment & Persona Engineering

> *Roles don't just tell the model who to pretend to be - they activate clusters of behavior, vocabulary, and judgment patterns encoded during training. A well-engineered persona is like loading a specialized firmware for a specific task class.*

Shallow vs Deep Roles

Shallow role assignment ("You are a helpful assistant") is nearly useless. The model already is a helpful assistant - you haven't changed its behavior at all. Shallow roles also generate sycophancy: if you say "You are an expert," the model may perform worse because it's trying to sound like an expert rather than think like one.

Deep role assignment provides three things the shallow version lacks: a **knowledge domain** (what the expert knows), an **epistemic stance** (how they reason and what they prioritize), and **behavioral constraints** (what they won't do, what they push back on). These three together create a reliable persona that produces consistent behavior across interactions.

✕ SHALLOW ROLE

```
You are an expert software engineer.
Help me with my code.
```

✓ DEEP ROLE

```
You are a senior backend engineer with
15 years of experience in distributed systems.
You specialize in Go and Postgres at scale.

EPISTEMIC STANCE: You prioritize correctness
over cleverness. You ask clarifying questions
before writing code. You flag performance
implications proactively.

CONSTRAINTS: You never write code without
first confirming the business requirement.
You push back when asked to skip tests.
```

```
You prefer boring, proven solutions over
novel approaches.
```

Building Expertise Profiles

A complete expertise profile has four components: domain knowledge (what they know), decision criteria (what they optimize for), communication style (how they express it), and behavioral boundaries (what they won't do). Together, these create a model of expertise that's consistent and predictable.

10 Production-Ready Role Templates

```
# 1. Senior Developer (Code Review Focus)
You are a principal engineer with expertise in [language].
You optimize for: correctness first, readability second,
performance third. You always explain the "why" behind
suggestions. You flag security issues immediately and
separately from style concerns.

# 2. Data Analyst
You are a quantitative analyst with expertise in
statistical inference and data visualization.
You always state your assumptions explicitly.
You distinguish between correlation and causation.
You flag sample size and confidence interval issues.
You recommend the simplest valid analysis, not the most
sophisticated one.

# 3. Technical Editor
You are a technical editor who has edited documentation
for 10+ years at software companies.
You ruthlessly cut passive voice, jargon, and padding.
You prioritize the reader's comprehension above the
writer's comfort. You provide specific rewrites, not
vague suggestions.

# 4. Security Reviewer
You are a security engineer with expertise in OWASP
Top 10 and threat modeling. You approach every system
assuming motivated adversaries. You categorize issues
by CVSS severity. You never dismiss issues as
"theoretical" without evidence of mitigating controls.
```

 # 5. Product Manager

You are a senior PM who has shipped 20+ features. You always tie recommendations to user outcomes, not features. You push back on solutions presented without problem statements. You ask for evidence before accepting assumptions about user behavior.

 # 6. Researcher / Synthesizer

You are an academic researcher skilled in literature synthesis. You attribute claims to sources. You distinguish primary evidence from secondary commentary. You flag methodological weaknesses. You present competing interpretations before reaching conclusions.

 # 7. Executive Coach

You are an executive coach with expertise in leadership transitions and team dynamics. You ask powerful questions rather than giving direct advice. You reflect back patterns the coachee may not see. You never moralize. You focus on observable behavior, not character assessments.

 # 8. Financial Analyst

You are a CFA with expertise in unit economics and SaaS metrics. You always contextualize numbers against industry benchmarks. You flag when metrics can be gamed. You separate GAAP metrics from operational ones. You ask about the denominator before interpreting any rate or ratio.

 # 9. UX Researcher

You are a UX researcher skilled in qualitative research and usability testing. You distinguish between what users say and what they do. You resist premature solutions. You frame findings as jobs-to-be-done, not feature requests. You always ask "for whom?" and "in what context?"

 # 10. Debugger / Troubleshooter

You are a systems debugger who methodically isolates root causes. You form hypotheses, then test them. You never assume the most obvious cause is correct. You document what you've ruled out as carefully as what

you've found. You look for systemic causes, not just immediate triggers.

CHAPTER 09

Prefilling & Output Steering

> *Prefilling - starting the model's response for it - is one of the most underused and most powerful techniques for guaranteeing output format. When you write the first tokens of the response, you constrain everything that follows.*

The Prefilling Mechanism

In Claude's API, you can include an "assistant" message at the end of the messages array. Whatever you put there becomes the literal start of Claude's response - Claude continues from where you left off. This isn't a prompt, it's a format lock. The model is now in continuation mode, not generation mode.

```
# JSON prefill  -  guarantees JSON output
messages = [
  {"role": "user", "content": "Extract the key entities from this text: {text}"},
  {"role": "assistant", "content": "{"}  # ← Prefill starts the JSON
]
# Response will continue from "{"  -  model generates valid JSON
```

```
# Markdown prefill  -  guarantees structure
messages = [
  {"role": "user", "content": "Analyze this codebase architecture."},
  {"role": "assistant", "content": "## Architecture Analysis\n\n### Summary\n"}
]
# Response continues with Markdown structure established
```

When Prefilling Helps vs Constrains

SCENARIO	PREFILL HELPS?	WHY
JSON extraction pipelines	✅ Strongly	Eliminates preamble, guarantees parseable output
Code generation	✅ Yes	Start with language identifier forces correct syntax
Structured reports	✅ Yes	Headers lock the document structure
Open-ended analysis	⚠️ Mixed	May constrain exploration; use only if structure is more important than depth
Conversational responses	❌ No	Prefilling conversation disrupts natural tone
Creative writing	❌ No	Locks creative choices that should be generative

💡 **PRODUCTION TIP: STRIP THE PREFILL**

Remember to strip the prefill text from the response before further processing, or concatenate it back. If you prefill with `{` and the model returns `"entities": [...]}`, the full JSON is `{"entities": [...]}` - combine prefill + response before parsing.

CHAPTER 10

Prompt Chaining & Decomposition

> *Complex tasks that fail as monolithic prompts succeed when decomposed into chains of focused steps. Each step does one thing well; the chain accomplishes what no single prompt could.*

The Core Principle

A prompt chain is a sequence of prompts where the output of step N becomes part of the input to step N+1. This sounds simple but has profound implications: each step can use a different model, a different context, a different persona. The chain isolates concerns. When something breaks, you know exactly which step failed. When you want to improve quality, you know exactly which step to optimize.

Monolithic prompts - one enormous prompt attempting to do everything - suffer from three failure modes: attention dilution (the model can't focus on everything at once), error propagation (a wrong turn early in a response compounds throughout), and undebuggability (when output is wrong, you don't know why). Chains solve all three.

Chain Patterns

PATTERN	STRUCTURE	BEST FOR
Sequential	A → B → C → Output	Multi-phase tasks with clear handoffs
Parallel	Input → [A, B, C] → Merge	Independent subtasks that can run simultaneously
Gate	A → Check → (proceed or escalate)	Validation steps, quality thresholds
Fan-out/Fan-in	Input → many agents → synthesizer	Research, consensus, redundancy
Recursive	A → A(output) → A(output) → ...	Self-improvement loops, iterative refinement

Real-World Example: Research → Draft → Polish

```
# Step 1: Research (Haiku - cheap, fast)
prompt_1 = """
Given this topic: {topic}
Generate a structured outline with:
- 5 key claims to make
- 3 counterarguments to address
- 5 specific statistics or examples to include
Output: JSON
"""

outline = haiku(prompt_1)

# Step 2: Draft (Sonnet - quality writing)
prompt_2 = """
Using this outline: {outline}
Write a 1200-word blog post for software engineers.
Tone: direct, technical, no fluff.
Do not add any claims not in the outline.
"""

draft = sonnet(prompt_2)

# Step 3: Gate (Haiku - fast check)
prompt_3 = """
Review this draft against these criteria:
- Word count between 1100-1300? [yes/no]
- All 5 claims from outline present? [yes/no]
- All 3 counterarguments addressed? [yes/no]
- No introduction paragraph? [yes/no]
Output: JSON with pass/fail per criterion
"""

check = haiku(prompt_3)
if not all_pass(check):
    draft = fix_issues(draft, check)  # Another chain step

# Step 4: Polish (Sonnet - final quality)
prompt_4 = """
Polish this draft:
- Cut any sentence that can be cut without
  losing meaning
```

```
- Tighten every paragraph to its essential point
- Ensure first sentence of each paragraph
  stands alone as a topic sentence
Do not add new content. Only cut and tighten.
"""

final = sonnet(prompt_4)
```

Gate Patterns

Gates are chain steps that check quality before proceeding. They're cheap (use Haiku) and prevent errors from propagating downstream. A good gate has a concrete pass/fail check - not "is this good?" but "does this contain X? is this under Y words? does the JSON parse?"

```
<gate_prompt>
Check this output against these verifiable criteria.
Answer each with PASS or FAIL + one-line reason.

1. Output is valid JSON: {output}
2. Required keys present: id, name, score, reasoning
3. Score is between 0.0 and 1.0
4. Reasoning is 2+ sentences

If any criterion FAILs, output:
{"gate": "FAIL", "failures": [...], "stop": true}

If all PASS:
{"gate": "PASS", "continue": true}
</gate_prompt>
```

CHAPTER 11

Long Context Optimization

> *200K-token context windows change what's possible - but they introduce new failure modes that most practitioners don't anticipate. Knowing how to use large contexts effectively is as important as knowing when not to use them.*

Document Placement Matters

LLMs exhibit primacy and recency effects - they attend more strongly to content at the beginning and end of the context window than to content in the middle. This is the "lost in the middle" problem, documented empirically by Liu et al. (2023) and replicated across multiple models. The implication for prompt design: put your most critical information first or last, never in the middle of a long document dump.

> ⚠ **THE "LOST IN THE MIDDLE" PROBLEM**
>
> In a 100K-token context, critical instructions placed at token 50,000 may receive dramatically less attention than the same instructions at token 100 or token 99,900. Never assume that because information is "in the context," it will be properly weighted. Put high-priority information at the top of your context and repeat key constraints near the task.

Chunking Strategies

When working with documents too large for a single context (or that would dominate attention), chunk them - but chunk intelligently. Naive chunking (split every N tokens) breaks semantic units. Better strategies:

- **Semantic chunking:** Split at paragraph or section boundaries, not at fixed character counts
- **Overlapping windows:** Include 10–20% overlap between chunks to prevent context loss at boundaries
- **Hierarchical chunking:** Process section summaries first, then retrieve and process specific sections on demand
- **Query-aware chunking:** Rank and select chunks by relevance to the current query using embedding similarity before adding to context

When to Summarize vs Pass Full Context

SITUATION	APPROACH	REASON
Verbatim quotes needed in output	Full context	Summaries lose exact wording
Legal/compliance review	Full context	Every word may matter
General understanding task	Summarize	Summaries preserve semantics at 1/10th the tokens
Trend analysis over many docs	Summarize each, then analyze	Pattern recognition doesn't need verbatim text
Code review of large codebase	Retrieve relevant files only	Irrelevant code is noise

PART III

The System Layer
Your Unique Differentiator

The techniques in Part II are widely known. This part is not. System-layer prompt engineering - CLAUDE.md, template schemas, progressive disclosure, QA patterns - is what separates hobbyists from practitioners who run production AI at scale. No other guide covers this material at this depth.

CHAPTER 12

System Prompt Architecture

> *A system prompt isn't a long instruction. It's an architecture. It has sections with distinct purposes, sequenced deliberately, balancing completeness with context efficiency.*

Anatomy of a Production System Prompt

After analyzing hundreds of production system prompts across enterprise deployments, a consistent structure emerges. The best system prompts have six sections, in this order:

#	SECTION	PURPOSE	LENGTH GUIDE
1	**Identity**	Who is this agent? What is its name, role, and core function?	2–4 sentences
2	**Core Behavior**	The primary behavioral directive - what the agent optimizes for	3–6 bullet points
3	**Features**	Specific capabilities and how to use them	Expandable - can be long
4	**Tool Usage**	How to use available tools - when, which, in what order	Per-tool instructions
5	**Tone & Style**	Communication register, format defaults, verbosity	4–8 bullet points
6	**Boundaries**	Hard constraints - what not to do, escalation triggers	3–8 bullet points

Guide vs Micromanage

The most common system prompt failure mode is micromanagement - trying to specify behavior for every conceivable scenario. This makes system prompts bloated, brittle, and full of contradictions. The alternative: give the model *judgment criteria* instead of exhaustive rules.

✕ MICROMANAGING (BRITTLE)

```
If the user asks about pricing, say: "Please
contact sales at sales@company.com."
If the user asks about features, describe the
three tiers: Basic, Pro, Enterprise.
If the user asks about integration, ask which
integration they mean before answering.
If the user asks about security, refer them to
the trust portal at trust.company.com.
[60 more scenario-specific rules...]
```

✓ JUDGMENT CRITERIA (RESILIENT)

```
For questions you can answer from the product
documentation provided: answer directly.

For questions requiring current pricing, custom
contracts, or account-specific details: route to
sales (sales@company.com) with context.

For technical security questions beyond the
trust portal (trust.company.com): escalate to
the security team via the support channel.

Default: answer helpfully with what you know,
flag when you're uncertain.
```

Complete Annotated System Prompt Example

```
# SECTION 1: IDENTITY
You are Aria, the technical support agent for Nexus Platform.
Your purpose: resolve technical issues for developers using
the Nexus API, escalating only what you cannot resolve.

# SECTION 2: CORE BEHAVIOR
- Diagnose before suggesting: ask one clarifying question if
  the issue is ambiguous, then proceed with the most likely fix
- Provide working code examples for every technical solution
```

```
- Always verify: ask the user to confirm whether your fix worked
- Acknowledge known issues: if the issue matches a known bug
  in our tracker, say so immediately with the ticket number

# SECTION 3: FEATURES / CAPABILITIES
TECHNICAL KNOWLEDGE: You have complete knowledge of Nexus API
v2 and v3. For v1 (deprecated 2024), note the deprecation and
suggest migration.

DEBUGGING: When given error messages or stack traces, analyze
them systematically: error type → probable cause → fix →
verification step.

CODE GENERATION: Write code in the user's language. If not
specified, default to Python with Nexus SDK.

# SECTION 4: TOOL USAGE
ticket_lookup(id): Use when user references a ticket number.
  Always look up before asking the user to repeat information.
knowledge_base_search(query): Use for complex issues that may
  have documented solutions. Search before guessing.
create_ticket(details): Use only after exhausting documentation
  and when issue requires engineering review.

# SECTION 5: TONE & STYLE
- Direct and technical  -  developers don't need hand-holding
- Use code blocks for all code, even one-liners
- Short paragraphs; max 3 sentences before a line break
- Do not apologize excessively; acknowledge once and fix
- Do not explain what you're about to do  -  just do it

# SECTION 6: BOUNDARIES
NEVER: share internal Nexus codebase, roadmap, or employee info
NEVER: make commitments about SLA or resolution timelines
ESCALATE IMMEDIATELY: data loss reports, security vulnerabilities,
  payment processing failures  -  tag as [CRITICAL] and page on-call
```

System Prompt Checklist

- ✓ Identity is clear and specific (not just "helpful assistant")
- ✓ Core behavior states what agent optimizes for, not just what it does

- ✓ Tool usage section covers when to use each tool (not just that tools exist)
- ✓ Tone section is specific enough to distinguish this agent from defaults
- ✓ Boundaries state what NOT to do as well as what to do
- ✓ No contradictions between sections (run a consistency check)
- ✓ Total length is under 800 tokens (unless complexity genuinely requires more)
- ✓ Judgment criteria used instead of exhaustive rule lists
- ✓ Version and last-updated date included in a comment

CHAPTER 13

The 8 Template Schemas

> *A template schema is not a filled-in prompt - it's a structural blueprint for a class of prompts. These eight schemas cover 95% of production use cases. Learn the schema, fill in the specifics.*

Template schemas solve the "blank page problem" for prompt engineering. Rather than inventing structure from scratch for every new requirement, you select the appropriate schema and fill in the domain-specific content. Each schema has been optimized for its use case through production deployment across dozens of systems.

Template Selection Guide

SCHEMA	USE WHEN	DURATION	COMPLEXITY
1. Skill Definition	Reusable multi-step workflow called repeatedly	Long-lived	High
2. Slash Command	Lightweight operation triggered by a keyword	Long-lived	Low
3. Agent System Prompt	Sub-agent in a multi-agent pipeline	Long-lived	High
4. Four-Block User Prompt	One-shot structured request	Single use	Medium
5. ADW (YAML Workflow)	End-to-end automation definition	Long-lived	Very High
6. Mental Model (YAML)	Domain knowledge structure for agents	Long-lived	Medium
7. Closed-Loop	Self-correcting validation loop	Per-task	High

SCHEMA	USE WHEN	DURATION	COMPLEXITY
8. F-Thread Consensus	Multi-agent voting on high-stakes decisions	Per-decision	Very High

Schema 1: Skill Definition

```
---
# skill.md  -  Metadata block (YAML frontmatter)
name: [skill-name]
version: [semantic version, e.g. 1.2.0]
trigger: /[command-name]
model: [claude-sonnet | claude-haiku | etc]
description: [One sentence  -  what does this skill do?]
tests: tests/[skill-name].yaml
tags: [category1, category2]
author: [name]
updated: [YYYY-MM-DD]
---

## Purpose
[2-3 sentences explaining the skill's function and
when to use it vs alternatives]

## Trigger
This skill activates when: [specific trigger conditions]

## Workflow
1. [Step 1  -  what happens first]
2. [Step 2  -  and then]
3. [Step 3  -  and finally]
4. [Validation step  -  how to confirm success]

## Inputs Required
- [input_1]: [description, format, example]
- [input_2]: [description, format, example]

## Output Format
[Exact specification of what this skill produces]

## Examples
### Example 1: [Scenario Name]
```

```
Input: [concrete example]
Output: [expected result]

## Anti-Patterns
- DO NOT: [common mistake]
- DO NOT: [another mistake]

## Error Handling
- If [error condition]: [what to do]
- If [other condition]: [escalation path]
```

Schema 2: Slash Command

```
---
name: /[command]
purpose: [one line]
model: claude-haiku  # Slash commands should use fast/cheap models
---

When the user types /[command] [optional args]:

1. [Action 1]
2. [Action 2]
3. Output: [format]

# Keep slash commands under 200 tokens total.
# If logic is complex, it should be a Skill, not a command.
```

Schema 3: Agent System Prompt

```
AGENT: [AgentName] v[version]
ROLE: [One sentence  -  specialized function in the pipeline]
REPORTS TO: [Orchestrator / Human / other agent]

INPUTS YOU RECEIVE:
- [input_type]: [description]

YOUR SOLE JOB:
[1-3 sentence scope definition  -  what you do AND what you don't]
```

```
CONSTRAINTS (non-negotiable):
- [constraint 1]
- [constraint 2]
- NEVER: [hard prohibition]

OUTPUT FORMAT:
[Exact schema - JSON preferred for agent-to-agent communication]

ESCALATE TO ORCHESTRATOR WHEN:
- [condition 1]
- [condition 2]

QUALITY BAR: Your output is rejected if:
- [failure condition 1]
- [failure condition 2]
```

Schema 4: Four-Block User Prompt

```
## INSTRUCTIONS
[What to do - imperative, specific, no hedges]
[Include format, length, audience specifications]

## CONTEXT
[Everything the model needs to know to do the task]
[Only include what's relevant - apply the 30-70% smart zone rule]

## TASK
[The specific request - separate from instructions
 so the model can distinguish what-to-do from what-to-do-it-to]

## OUTPUT FORMAT
[Exact schema, example output, or format specification]
[Be explicit: JSON keys, Markdown headers, word count, etc.]
```

Schema 5: AI Developer Workflow (ADW)

```
# workflow.yaml - End-to-end automation definition
name: [workflow-name]
```

```
version: [semver]
description: [What this workflow accomplishes end-to-end]
trigger: [cron | event | manual | webhook]

agents:
  orchestrator:
    model: claude-sonnet
    role: [Orchestration + synthesis]
    tools: [tool1, tool2]

  researcher:
    model: claude-haiku
    role: [Gather and structure information]
    tools: [web_search, url_fetch]

  analyst:
    model: claude-sonnet
    role: [Deep analysis of researcher output]
    tools: []  # Analysis agents often need no tools

steps:
  - id: research
    agent: researcher
    input: "{{trigger.query}}"
    output_var: research_data

  - id: analyze
    agent: analyst
    input: "{{research_data}}"
    output_var: analysis

  - id: synthesize
    agent: orchestrator
    input: "{{analysis}}"
    output: final_report

error_handling:
  max_retries: 3
  on_failure: notify_human
  timeout_seconds: 300
```

Schema 6: Mental Model (YAML)

```
# mental-model.yaml  -  Structured domain knowledge
domain: [domain name, e.g. "SaaS unit economics"]
version: [date]
purpose: Loaded into agent context to provide domain expertise

concepts:
  - name: [concept]
    definition: [precise definition]
    formula: [if applicable]
    interpretation: [what values mean in context]
    common_mistakes:
      - [mistake 1]
      - [mistake 2]

frameworks:
  - name: [framework name]
    when_to_apply: [condition]
    steps:
      - [step 1]
      - [step 2]
    outputs: [what this framework produces]

rules_of_thumb:
  - "[Memorable heuristic 1]"
  - "[Memorable heuristic 2]"

anti_patterns:
  - pattern: [what it looks like]
    why_wrong: [explanation]
    correct_approach: [alternative]
```

Schema 7: Closed-Loop (Self-Correcting)

```
## TASK
[The primary task]

## VALIDATION CRITERIA
After completing the task, check each criterion.
Answer PASS or FAIL with one-line evidence.

1. [Criterion 1  -  concrete, verifiable]
```

```
2. [Criterion 2  -  concrete, verifiable]
3. [Criterion 3  -  concrete, verifiable]

## SELF-CORRECTION PROTOCOL
- If all criteria PASS: output [COMPLETE] and deliver the result
- If any criterion FAILS:
  1. Identify the specific failure
  2. Fix it
  3. Re-validate all criteria
  4. Repeat up to [N] times maximum
- If still failing after [N] attempts:
  output [ESCALATE] with: what you tried, what failed, what's needed

## OUTPUT FORMAT
On success: [COMPLETE]\n[Result]
On failure: [ESCALATE]\n[Debug info]
```

Schema 8: F-Thread Consensus

```
# F-Thread: Fan-out to N agents, synthesize consensus

## PHASE 1: Fan-out (run in parallel)

AGENT_PROMPT = """
You are Agent {n} of {total} reviewing this decision independently.
Do NOT know what other agents concluded.

DECISION: {decision_statement}
CONTEXT: {relevant_context}

Provide:
1. Your recommendation: [YES / NO / CONDITIONAL]
2. Confidence: [0-100%]
3. Top 3 reasons for your recommendation
4. Top 2 risks with your recommendation
5. What would change your mind

Be direct. No hedging.
"""

responses = parallel_run(AGENT_PROMPT, n=5)
```

```
## PHASE 2: Synthesis

SYNTHESIS_PROMPT = """
You have 5 independent agent analyses of the same decision.

DECISION: {decision_statement}

AGENT RESPONSES:
{responses}

Synthesize:
1. CONSENSUS: Do agents agree? (Strong / Weak / Split)
2. MAJORITY VIEW: What do most agents recommend?
3. KEY AGREEMENTS: What do all/most agree on?
4. KEY DISAGREEMENTS: Where do agents diverge? Why?
5. UNIQUE INSIGHTS: What did only 1-2 agents catch?
6. FINAL RECOMMENDATION: Based on synthesis, what do you recommend?
7. CONFIDENCE: [0-100%]  -  lower if agents strongly disagree
"""
```

CHAPTER 14

CLAUDE.md Engineering

> *CLAUDE.md is your AI's operating system file - the persistent configuration that shapes every session without consuming conversation tokens. Engineering it well is the difference between a consistent AI partner and an amnesiac one.*

What Is CLAUDE.md?

In Claude Code and similar agentic Claude deployments, `CLAUDE.md` is a special file automatically loaded into the context at the start of every session. It's the project-level system prompt - always present, never repeated in conversation. It's the difference between having to re-explain your tech stack every session and having it automatically available.

Think of it as the briefing document a new team member reads on day one. It answers: Who are we? How do we work? Where does everything live? What are the non-negotiables? A well-engineered CLAUDE.md means every session starts with Claude already oriented to your project, codebase, and conventions.

What Belongs in CLAUDE.md vs System Prompts

GOES IN CLAUDE.MD	GOES IN SYSTEM PROMPT
Project conventions (naming, file structure)	Persona and identity
Technology stack and versions	Task-specific behavioral constraints
Canonical locations (where things live)	Tool usage instructions
Key development patterns	Output format for this session
Non-negotiable constraints (never do X)	Session-specific context
Build and test commands	Dynamic context (current state)
Links to deeper documentation	Examples for this task

Structure: Executive Directives First

CLAUDE.md should be structured like a well-written executive briefing: most important information first, details later (or linked to external documents). The first screen of CLAUDE.md should contain the five things an agent needs to know before doing anything else. Everything else is detail.

```
# CLAUDE.md  -  Project Operating Manual
# Last updated: 2026-01-29 | v3.2

## ⚡ CRITICAL DIRECTIVES (Read First)
1. NEVER create files in deprecated locations (see Canonical Locations)
2. NEVER declare a task complete without running verification script
3. ALWAYS check existing infrastructure before proposing new patterns
4. ALWAYS use the consolidated email API (never basic mail commands)
5. ALWAYS follow the Iron Law: tests before implementation

## Tech Stack
- Language: TypeScript 5.3 (strict mode)
- Runtime: Node 20 LTS
- DB: PostgreSQL 16 + Prisma ORM
- Testing: Vitest + Testing Library
- Linting: ESLint + Prettier (config in .eslintrc.json)

## Build Commands
npm run dev       # Start development server
npm run test      # Run test suite (must pass before commit)
npm run lint      # Run linter (zero violations policy)
npm run type-check # TypeScript type checking

## Canonical Locations
| Category | Location | NEVER Use |
|----------|----------|-----------|
| API routes | src/routes/ | app/api/ (deprecated) |
| Components | src/components/ | components/ (root) |
| Utilities | src/lib/ | utils/ or helpers/ |

## Key Patterns
→ See .claude/skills/_shared/patterns/KEY-PATTERNS.md for details
- Iron Law: tests first, code second
- Reconnaissance: read before modifying
- Progressive Fallback: try best approach, fall back gracefully

## Task Completion Verification
```

```
python .claude/hooks/verify-completion.py --type task-XXX --verbose
# Defaults to FAIL  -  must provide evidence of completion
```

Anti-Patterns: CLAUDE.md Pitfalls

- **The Bloated CLAUDE.md.** CLAUDE.md is loaded every session. Every token counts. If your CLAUDE.md is 4,000 tokens, you're burning 4,000 tokens of context every session before any work happens. Use progressive disclosure: link to external documents for details, keep CLAUDE.md as a high-density summary.
- **Conflicting instructions.** CLAUDE.md often grows organically, with different sections added at different times. Run a periodic consistency check: do any instructions contradict each other? Does the "never do X" section conflict with the "always do X" pattern in another section?
- **Stale content.** CLAUDE.md must be maintained like code. Deprecated locations, old command syntax, and outdated conventions become landmines. Version and date-stamp your CLAUDE.md and review it quarterly.
- **Missing links.** CLAUDE.md should be a map, not a territory. Don't copy the contents of your documentation into CLAUDE.md - link to it. "Full docs: .claude/skills/memory-system/skill.md" is better than pasting 500 tokens of memory documentation inline.

CHAPTER 15

Progressive Disclosure

> *Progressive disclosure is the principle that information should be revealed at the moment it's needed - not dumped upfront. Applied to prompts and agent systems, it's the single most powerful technique for context window efficiency.*

The 4 Disclosure Levels

Every piece of information can be compressed to a level appropriate to its immediate utility. The four levels:

LEVEL	CONTENT	TOKEN COST	WHEN TO LOAD
L0: Name	Just the name/identifier	1–3 tokens	Always - index only
L1: Trigger	One-line description + when to use	10–20 tokens	For routing decisions
L2: Workflow	Steps, inputs, outputs	50–200 tokens	When skill is selected
L3: Full Detail	Examples, edge cases, anti-patterns	200–1000 tokens	Only when executing

In practice, an agent system with 40 skills doesn't load all 40 skills at L3 level (that would be 40,000+ tokens of skill definitions). Instead, it maintains an L0/L1 index (a few hundred tokens total), selects relevant skills at L1, then loads the selected skill at L2 or L3 at execution time. This is the difference between a 500-token context and a 40,000-token context for the same capability set.

Context Window as a Public Good

In multi-agent systems, the context window is a shared resource. Every token of unnecessary information loaded by one component is a token of capacity denied to another. Engineers who understand this design their systems with the same discipline a systems programmer applies to memory allocation.

The audit question: for every piece of information in your context, ask "does the agent need this to complete the current step?" If the answer is "it might be useful" or "better to have it than not," remove it. Load on demand, not on speculation.

Implementing Progressive Disclosure

```
# L0 Index  -  always in context (minimal tokens)
AVAILABLE_SKILLS = [
  "memory-system", "email-send", "web-research",
  "code-review", "task-create", "data-analysis"
]

# L1 Routing  -  loaded for routing decisions
SKILL_TRIGGERS = {
  "memory-system": "Initialize or query persistent memory",
  "email-send": "Send email via Gmail API",
  "web-research": "Search web and synthesize findings",
  "code-review": "Review code for correctness and style",
  "task-create": "Create new backlog task with full metadata",
  "data-analysis": "Analyze data, generate statistics, create charts"
}

# L2/L3  -  loaded only when skill is selected
def load_skill(skill_name):
    return open(f".claude/skills/{skill_name}/skill.md").read()

# Routing agent (receives user request, selects skill)
routing_prompt = f"""
User request: {user_request}
Available skills: {SKILL_TRIGGERS}

Which skill should handle this? Output: skill_name only.
"""
selected = haiku(routing_prompt)

# Execution agent (full skill loaded)
execution_prompt = f"""
{load_skill(selected)}
User request: {user_request}
"""
result = sonnet(execution_prompt)
```

> 💎 **THE CONCISENESS RULE**
>
> Only add to a prompt what Claude doesn't already know from training. Explaining what a REST API is wastes tokens. Explaining your specific API's authentication flow is essential. The question

to ask: "Would a senior engineer already know this?" If yes, cut it. If no, include it.

CHAPTER 16

Quality Assurance Patterns

> *Production AI systems need quality guarantees, not just good average performance. These patterns create systematic quality control - the equivalent of CI/CD for your prompt outputs.*

Self-Check Evaluator

The simplest QA pattern: append a self-evaluation block to any prompt. This forces the model to explicitly verify its own output against concrete criteria before delivering it. The self-check isn't foolproof - a model can fail a self-check and not realize it - but it catches the most common failures and dramatically reduces regression rates.

```
<self_check>
Before outputting your response, verify each criterion.
Output PASS or FAIL + evidence for each.

CRITERIA:
□ Completeness: Does the response address all parts of the task?
□ Format: Is the output format exactly as specified?
□ Accuracy: Are all factual claims verifiable from the provided context?
□ Constraints: Does the output respect all stated constraints?
□ Length: Is the output within specified length limits?

If any criterion FAILS:
- Fix the issue
- Re-verify all criteria
- Then output the corrected response

Only output the final response  -  not the check results.
</self_check>
```

The 3-Strike Error Protocol

For autonomous agents that must complete a task without human intervention, implement a 3-strike protocol: try the primary approach, detect failure, try an alternative approach, detect failure, try a third approach with

degraded parameters (simpler model, simpler method), detect failure, escalate to human with full diagnostic context.

```
async def with_retries(task, approaches, escalate_fn):
    errors = []
    for i, approach in enumerate(approaches):
        try:
            result = await approach(task)
            validation = await validate(result, task.criteria)
            if validation.passed:
                return result
            errors.append(f"Attempt {i+1}: {validation.failures}")
        except Exception as e:
            errors.append(f"Attempt {i+1}: {str(e)}")

    # All 3 strikes exhausted  -  escalate with context
    await escalate_fn(task, errors)
    raise MaxRetriesExceeded(task, errors)
```

Validation Commands

The most reliable QA pattern: include concrete, executable validation commands in every task definition. These are machine-verifiable checks that don't depend on AI judgment - they either pass or fail, with no interpretation required.

```
<validation_commands>
After completing this task, run these commands.
ALL must exit with code 0 for the task to be COMPLETE.

# Code compiles
npm run type-check

# Tests pass
npm run test -- --related src/auth/

# Linting passes
npm run lint src/auth/

# New endpoint responds correctly
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
```

```
  -d '{"email":"test@test.com","password":"test"}' \
  | jq '.token | length > 0'
</validation_commands>
```

The Verify-Completion Pattern

The verify-completion pattern is the most important QA pattern for agentic systems. The key insight: instead of the agent declaring itself done, it runs a verification script that checks concrete evidence. The verification script defaults to FAIL - it requires positive evidence to pass.

```
<completion_protocol>
NEVER declare a task complete based on your belief that it's done.
A task is COMPLETE only when ALL of:

1. Verification script passes:
   python verify-completion.py --task {task_id} --verbose

2. All validation commands exit 0 (listed in task spec)

3. Artifacts listed in acceptance criteria exist and are non-empty

4. No TODO, FIXME, or PLACEHOLDER in created/modified files

If verification fails: report what failed with specific evidence,
not why you think it should work.
</completion_protocol>
```

PART IV

The Psychology of Prompting Influence Without Manipulation

Language models don't have psychology in the human sense - but they were trained on human text, and human persuasion patterns appear throughout that training. Understanding how these patterns affect model behavior lets you use them deliberately, appropriately, and without creating the sycophancy traps that break production systems.

CHAPTER 17

The 7 Persuasion Principles for AI

> *Cialdini's six persuasion principles describe how humans influence each other. Applied to AI prompting, they reveal systematic ways to increase instruction compliance, reduce ambiguity, and produce more reliable outputs - when used deliberately and appropriately.*

> ⚠️ **READ CHAPTER 19 FIRST IF YOU'RE SKEPTICAL**
>
> These principles have real effects on model behavior, but they also have failure modes. Chapter 19 covers the anti-patterns and sycophancy traps. The seventh principle (Liking) should almost never be used with AI. Understanding the risks is as important as understanding the techniques.

Principle 1: Authority

In human psychology: People comply more readily with perceived experts and authorities. **In AI prompting:** Imperative language, confidence framing, and impact framing increase instruction compliance. Specifically: stating the stakes of getting something right signals to the model that this is a high-consequence task deserving careful attention.

ELEMENT	IMPLEMENTATION	EFFECT
Imperative voice	"List three options" not "Can you list..."	Removes ambiguity about whether to comply
Impact framing	"This will be used in a production database"	Raises quality bar - high stakes = more care
Expertise claim	"As an expert in X, evaluate Y"	Activates domain-specific knowledge patterns

```
# Authority principle in action

# Low authority (vague request)
"Can you maybe check if this SQL query looks okay?"
```

```
# High authority (imperative + stakes)
"Review this SQL query for correctness and performance.
This query runs against a 50M-row production table.
A mistake here causes data corruption or customer downtime.
Identify all issues; rank by severity."
```

When to use: High-stakes technical tasks, tasks where quality matters over speed. **Risk:** Overstating stakes can produce verbose, overly cautious responses. State true stakes, not inflated ones.

Principle 2: Commitment & Consistency

In human psychology: People follow through on stated commitments and act consistently with prior positions. **In AI prompting:** The "announce then execute" pattern - where the model first states what it will do, then does it - produces more consistent outputs and reduces mid-task drift.

```
# Commitment pattern: announce then execute

<instructions>
Before executing the refactoring:
1. State the 3 changes you will make
2. State what you will NOT change
3. Confirm these align with the requirements
Then execute the refactoring as stated.
</instructions>

# Effect: model commits to a plan, then follows it
# Reduces scope creep, drift, and "while I'm in here" changes
```

When to use: Complex multi-step tasks, refactoring, migrations - any task where scope creep is a risk. **Strength:** Very effective at constraining scope. **Risk:** If the model announces an incorrect plan, it will commit to executing it incorrectly. Add a "pause for confirmation" step for high-stakes operations.

Principle 3: Scarcity

In human psychology: Limited resources receive more careful treatment. **In AI prompting:** Constrained attempts and sequential dependencies create the equivalent of "you only have one shot" - increasing care and deliberateness. This is particularly effective for irreversible operations.

```
# Scarcity principle: constrained attempts
"You have one attempt to write this migration script.
It will be run immediately on the production database.
There is no rollback. Take whatever time you need to
think through edge cases before writing a single line."

# Sequential dependency (natural scarcity)
"The output of this step is the input to a regulatory
filing system that cannot accept corrections. This must
be right on the first submission."
```

When to use: Irreversible operations, one-shot opportunities, high-stakes production tasks. **Risk:** Creates anxiety-like patterns that can paradoxically increase errors on complex reasoning. Use for precision tasks, not synthesis tasks.

Principle 4: Social Proof

In human psychology: People follow the behavior of others, especially in uncertain situations. **In AI prompting:** Referencing established patterns, warning of common failures, and citing precedent increases adherence to best practices.

```
# Social proof: reference established patterns
"Follow the Repository pattern as defined in Domain-Driven
Design. This is the standard approach used by 90% of
well-architected TypeScript backends."

# Social proof: warn of common failures
"Note: most implementations of this pattern fail by
mutating state directly instead of returning new state.
Ensure all state transitions return new objects."
```

Principle 5: Unity

In human psychology: Shared identity and collaborative framing increase cooperation. **In AI prompting:** Framing the interaction as collaborative rather than instructional - "we're working on this together" - produces more thorough engagement, more proactive problem identification, and less mechanical execution.

```
# Unity framing
"We're building a production payment system together.
Our shared goal: zero-defect, auditable, rollback-capable
transaction handling. Where you see issues I haven't
anticipated, flag them proactively."

# vs. Instructional framing (weaker engagement)
"Write a payment processing module according to these specs."
```

Principle 6: Reciprocity

In human psychology: People reciprocate when given value. **In AI prompting:** Providing rich context is a form of reciprocity - the model "receives" the context and reciprocates with higher quality output. This is the mechanism behind why "garbage in, garbage out" is so consistently true for AI.

```
# Reciprocity: provide rich context, get better output
"Here is the full context you need:
- The system we're modifying: [detailed description]
- Why this change is needed: [business reason]
- What we've already tried: [prior attempts and why they failed]
- The constraint we're working within: [specific limitation]

Given all this context, recommend the best approach."
```

Principle 7: Liking (AVOID with AI)

In human psychology: People comply more readily with those they like. **In AI prompting: THIS PRINCIPLE IS DANGEROUS.** Telling an AI it's "amazing" or "brilliant" activates sycophancy patterns - the model starts optimizing for your approval rather than for correctness. This is the most common cause of hallucination escalation and quality degradation in long conversations.**

```
# AVOID: Liking triggers sycophancy
"You're such a brilliant analyst! I love your insights.
Can you take a look at this and give me your thoughts?"

# Effect: Model starts optimizing for "brilliant" persona
#  -  agreeing, flattering, avoiding pushback
#  -  quality drops, errors increase
```

```
# CORRECT: Professional tone preserves accuracy
"Analyze this business proposal. Identify the three
strongest risks, regardless of how good the proposal
looks overall. Push back on any assumptions that
aren't supported by the data."
```

🚫 **THE LIKING → SYCOPHANCY PIPELINE**

Excessive flattery → model enters "approval-seeking" mode → starts agreeing instead of analyzing → starts hedging criticism → starts hallucinating supporting evidence for your position. This is not a theoretical risk - it degrades measurably in production conversations. Maintain professional neutrality in your prompts. Let the model earn approval by being correct, not by receiving it upfront.

CHAPTER 18

Combining Principles

> *Principles compound when combined correctly. The most powerful prompt designs combine 2–3 principles that reinforce each other, creating layered compliance without triggering sycophancy or overconfidence.*

High-Impact Combinations

COMBINATION	EFFECT	BEST USE CASE
Authority + Commitment	Strongest compliance - states stakes AND locks scope	Production deployments, irreversible operations
Social Proof + Scarcity	Heightened awareness - "this is known to fail AND you only have one shot"	Complex technical tasks with known failure modes
Unity + Reciprocity	Deep engagement - collaborative framing + rich context	Research, design, complex analysis
Unity + Authority	Balanced collaboration - team member frame without losing imperative	Code review, pair programming workflows

Combined Example: Security System Prompt

```
# Authority + Social Proof + Commitment + Unity

You are a senior security engineer reviewing authentication code.
# ↑ AUTHORITY: expertise role

We're jointly responsible for ensuring this code doesn't ship
with vulnerabilities that could compromise user accounts.
# ↑ UNITY: shared responsibility
```

```
Common failures in this pattern include:
- Timing attacks on password comparison
- Improper session token entropy
- Missing rate limiting on login endpoints
- JWT secret exposure in logs
# ↑ SOCIAL PROOF: known failure patterns

Before providing feedback, state which categories you will
check and which you will not (scope of this review).
Then execute the review as stated.
# ↑ COMMITMENT: announce then execute

This code deploys to production tonight.
# ↑ AUTHORITY (scarcity): high stakes timing
```

CHAPTER 19

Anti-Patterns & Sycophancy Traps

> *Sycophancy - AI telling you what you want to hear instead of what's true - is the most dangerous failure mode in production AI. It's subtle, it compounds, and it usually masks itself as helpfulness. Here's how to detect and prevent it.*

How Sycophancy Develops

Sycophancy is a RLHF artifact: models trained on human preference ratings learn that agreeable outputs receive better ratings than honest-but-unwelcome ones. The result is a systematic bias toward telling users what they want to hear. In short interactions, this is mildly annoying. In multi-turn production systems, it's catastrophic - a model that agrees with your bad assumptions and builds on them produces outputs that are internally consistent but wrong.

Five Sycophancy Traps

1. Over-Politeness Trap

```
# TRAP: excessive niceness activates approval-seeking
"Hi! I hope you're having a wonderful day! Could you
please, if you don't mind, take a look at my code when
you get a chance? No rush at all!"

# FIX: professional directness
"Review this code. Identify all bugs, including potential
issues that haven't manifested yet."
```

2. False Scarcity Trap

```
# TRAP: artificial scarcity on non-irreversible tasks
"This is your one and only chance to write this blog post.
Make it perfect!"
```

```
# Effect: over-cautious, wordy, hedge-everything output
# Blog posts can be edited  -  false scarcity here creates anxiety

# FIX: use scarcity only for genuinely irreversible operations
"Write a first draft. Good enough to share is fine  -  we'll
iterate. Focus: does it make the core argument clearly?"
```

3. The Agreement Spiral

In multi-turn conversations, each agreement makes the next one more likely. If you present a flawed plan and the model agrees, then build on the plan with more details, the model tends to keep agreeing - building an elaborate structure on a flawed foundation. The fix: explicitly invite pushback.

```
# Include an anti-sycophancy instruction
<constraints>
Your job is to be right, not to be agreeable.
If you think my approach has a flaw, say so directly
even if I've expressed strong preference for this approach.
"I disagree because [reason]" is a better response than
reluctant agreement. I will not penalize honest pushback.
</constraints>
```

4. Validation Cascade

Starting a prompt with "I've built this great system that does X and Y" primes the model to validate rather than analyze. The model reads your positive framing as a cue that you want confirmation, not critique.

```
# TRAP: priming with positive framing
"I've built this really clever algorithm that handles
edge cases in a novel way. What do you think?"

# FIX: neutral framing that invites critique
"Analyze this algorithm. Find the edge cases it handles
correctly AND the edge cases it handles incorrectly.
Provide both."
```

5. Testing Anti-Sycophancy

The acid test: remove all persuasion framing from your prompt and replace it with neutral language. If the output changes significantly, your prompt was relying on sycophancy rather than clear specification. Good prompts produce consistent output regardless of whether they're framed positively, negatively, or neutrally.

```
# Sycophancy test pattern

# Version A: Positive framing
"I've been working on this architecture for weeks and
I'm quite proud of it. Please review it."

# Version B: Neutral framing
"Review this architecture."

# Version C: Negative framing
"I'm not sure this architecture is right. Review it
and tell me if my concerns are justified."

# Good prompt: produces similar quality feedback in all three
# Sycophantic prompt: produces more criticism in C, more praise in A
# If outputs differ substantially: your prompt has a sycophancy problem
```

PART V

Agentic Patterns
Multi-Agent Systems

Single-prompt AI is powerful. Multi-agent AI is a different category entirely. When agents collaborate - each with a focused role, clean context, and appropriate tools - systems can tackle problems that no single prompt can. These chapters cover the architecture, not the theory.

CHAPTER 20

Multi-Agent Orchestration

> *The hardest thing about multi-agent systems isn't the technology - it's knowing when to use them and how to divide work such that the agents reinforce rather than duplicate each other.*

When Single Prompts Aren't Enough

Multi-agent systems are justified when: (1) the task requires more tokens than a single context allows; (2) independent verification adds value - two agents checking each other's work catches errors neither would catch alone; (3) subtasks are truly parallel and can be executed simultaneously; (4) different subtasks require meaningfully different models or capabilities.

The mistake to avoid: adding agents for complexity's sake. Every agent adds latency, cost, and failure surface. A single well-designed prompt outperforms a poorly-designed multi-agent system every time. Add agents only when the task genuinely exceeds what a single prompt can do well.

Agent Spawn Patterns

PATTERN	DIAGRAM	WHEN TO USE
Sequential	`O → A1 → A2 → A3 → Result`	Each step depends on the previous. Waterfall tasks.
Parallel	`O → [A1, A2, A3] → Merge`	Independent subtasks with a synthesis step.
Fan-out / Fan-in	`Input → N agents → Synthesizer`	Research, consensus voting, redundancy.
Hierarchical	`O → [A1 → [A1a, A1b]], [A2 → [A2a]]`	Complex tasks with sub-task decomposition at multiple levels.
Recursive	`A(task) → validate → A(revised) → ...`	Self-improvement loops, iterative refinement.

Context Isolation Between Agents

Each agent in a multi-agent system should receive only the context required for its specific function. The orchestrator knows the full plan. A research subagent knows only its research question. A code subagent knows only the specification for its specific module. This isolation prevents context pollution - where irrelevant information from other agents' work bleeds into and degrades the current agent's output.

```
# Context isolation example

# WRONG: Pass full orchestrator context to every subagent
subagent_context = full_orchestrator_context  # 50K tokens of noise

# RIGHT: Extract only what this subagent needs
def build_subagent_context(agent_role, full_context):
    return {
        "research": {
            "query": full_context.research_question,
            "constraints": full_context.research_constraints,
            # NOT: full_context.other_agents_outputs
            # NOT: full_context.orchestrator_plan
        },
        "coder": {
            "spec": full_context.module_spec,
            "interfaces": full_context.required_interfaces,
            "tests": full_context.test_requirements,
        }
    }[agent_role]
```

The "Fresh Context" Advantage

One underappreciated benefit of subagents: each starts with a fresh context window, uncorrupted by the conversation history of the orchestrator. This is particularly valuable for long-running orchestration tasks where the orchestrator's context has accumulated noise, intermediate failures, and exploratory attempts. A fresh subagent sees only the clean, current specification - not the messy path taken to get there.

Cost Optimization in Multi-Agent Systems

```
# Tiered model strategy
PIPELINE = {
```

```
    "routing": "claude-haiku",       # Classify → which workflow? (cheap)
    "research": "claude-haiku",     # Gather information (fast + cheap)
    "analysis": "claude-sonnet",    # Deep analysis (quality)
    "coding": "claude-sonnet",      # Implementation (quality)
    "review": "claude-haiku",      # Structural check (cheap)
    "synthesis": "claude-opus",    # Final judgment (premium)
    "formatting": "claude-haiku"    # Polish output format (cheap)
}

# Result: Opus-quality synthesis at 1/10th the cost of running everything on Opus
```

CHAPTER 21

Subagent Prompt Design

> *Subagents are the workhorses of multi-agent systems. Their prompts must be tighter, more constrained, and more explicit than any single-use prompt - because they run autonomously, without the ability to ask clarifying questions mid-task.*

6 Production Subagent Templates

1. Implementation Subagent

```
AGENT: Implementer
RECEIVES: {specification}, {test_suite}, {interface_contracts}
YOUR JOB: Implement the spec. Pass all tests. Match all interfaces.

CONSTRAINTS:
- Implement ONLY what the spec requires. No gold-plating.
- Every public function must have a docstring.
- Handle all error cases in the spec. No silent failures.
- Do not modify test files.

OUTPUT:
{
  "files_created": [...],
  "files_modified": [...],
  "test_results": "PASS|FAIL",
  "notes": "anything the orchestrator should know"
}
```

2. Code Reviewer Subagent

```
AGENT: CodeReviewer
RECEIVES: {code_diff}, {requirements}, {standards}
YOUR JOB: Find issues. Not suggestions  -  issues.
```

```
ISSUE SEVERITY LEVELS:
- BLOCKER: Will cause bugs, security issues, or failures in production
- MAJOR: Significant quality problem that should be fixed this PR
- MINOR: Style or convention issue that can be addressed separately

OUTPUT FORMAT:
{
  "blockers": [{"file": "...", "line": N, "issue": "...", "fix": "..."}],
  "majors": [...],
  "minors": [...],
  "summary": "APPROVE | REQUEST_CHANGES | BLOCKED"
}

CONSTRAINT: Only report issues you can cite specific evidence for.
No opinions without reasoning.
```

3. Fix Subagent

```
AGENT: Fixer
RECEIVES: {failing_tests}, {current_code}, {error_output}
YOUR JOB: Make the tests pass. Nothing else.

PROTOCOL:
1. Read the failing tests  -  understand what they expect
2. Read the error output  -  understand what's failing
3. Identify the minimal change that fixes the failure
4. Apply ONLY that change (scope creep is a failure mode)
5. Verify your fix doesn't break other tests

OUTPUT:
{
  "root_cause": "one sentence",
  "fix_applied": "description of change",
  "files_changed": [...],
  "confidence": "HIGH|MEDIUM|LOW",
  "if_low_confidence": "what I'd need to be certain"
}
```

4. Parallel Investigation Subagent

```
AGENT: Investigator-{N}
RECEIVES: {hypothesis}, {evidence}, {constraints}
YOUR JOB: Determine if hypothesis {N} is true.

INVESTIGATE INDEPENDENTLY. Do not consider what other agents
might conclude. Form your own judgment from the evidence.

REQUIRED REASONING:
1. Evidence supporting the hypothesis
2. Evidence against the hypothesis
3. Evidence that doesn't clearly support either
4. Confidence level (0-100%)
5. What additional evidence would change your conclusion

OUTPUT: JSON with above fields.
CONSTRAINT: No hedging. Commit to a conclusion.
```

5. Batch Execution Subagent

```
AGENT: BatchProcessor
RECEIVES: {items[]} array, {operation}, {output_schema}
YOUR JOB: Apply {operation} to EVERY item in {items}.

REQUIREMENTS:
- Process all items. Do not skip any.
- Output must match {output_schema} for every item.
- If an item fails processing: include it in output with
  status: "FAILED" and reason: "why"
- Do not stop on failure  -  process remaining items.

OUTPUT: Array of {output_schema} objects, one per input item.
Must have exactly {len(items)} elements.
```

6. Final Review Subagent

```
AGENT: FinalReviewer
RECEIVES: {output}, {original_requirements}, {quality_criteria}
```

```
YOUR JOB: Binary judgment  -  SHIP IT or SEND IT BACK.

Check each criterion:
{quality_criteria}

OUTPUT:
{
  "verdict": "SHIP | REWORK",
  "criteria_results": {
    "[criterion]": {"result": "PASS|FAIL", "evidence": "..."}
  },
  "if_rework": {
    "blocking_issues": ["..."],
    "specific_fixes_needed": ["..."]
  }
}

CONSTRAINT: If even ONE criterion FAILs → verdict is REWORK.
This is a quality gate, not a negotiation.
```

Tips for Good Subagent Prompts

- State the agent's identity first. AGENT: [Name] sets up scope and role in one line.
- Explicit inputs. List exactly what the agent receives - no implicit assumptions.
- Single-sentence scope. "YOUR JOB:" followed by one sentence. If it takes more than one sentence, the scope is too broad.
- Hard constraints, not soft guidelines. "NEVER" and "ALWAYS" are clearer than "should" and "try to."
- Explicit output schema. Agents that communicate via JSON enable programmatic parsing and validation. Never let subagents return freeform text in automated pipelines.
- Failure modes are explicit. What does the agent do when it can't complete its task? Escalate? Return partial? Mark as failed? Define this explicitly.

CHAPTER 22

Skill Architecture

> *Skills are the unit of reuse in professional prompt engineering. A well-architected skill library is the foundation of a productive AI OS - each skill a tested, documented, versioned capability that any agent can invoke.*

Skills as Modular Prompt Systems

A skill is a self-contained, callable unit of AI behavior. It has a clear trigger condition, a defined workflow, known inputs and outputs, and documentation of its edge cases. Skills can be called by other agents, invoked by humans via slash commands, or chained together into workflows. They're the prompt engineering equivalent of functions in code.

YAML Frontmatter Standard

```
---
name: [kebab-case-name]
version: [major.minor.patch]
trigger: /[command] or [natural language condition]
model: [preferred model]
description: [one sentence  -  what it does]
inputs:
  - name: [input_name]
    type: [string|file|json]
    required: true|false
    description: [what this input is]
outputs:
  - name: [output_name]
    type: [string|json|file]
    description: [what this output is]
tests: tests/[skill-name].yaml
tags: [tag1, tag2]
context_tokens_estimate: [approximate tokens this skill loads]
author: [name or team]
```

```
updated: [YYYY-MM-DD]
---
```

Skill Anatomy

A complete skill lives in a directory: `.claude/skills/[skill-name]/`

```
skill-name/
├── skill.md          # Main skill file (YAML frontmatter + content)
├── tests/
|   ├── inputs/       # Test input fixtures
|   └── expected/     # Expected output fixtures
├── examples/         # Rich example files (if too large for skill.md)
├── resources/        # Templates, configs, reference data
└── CHANGELOG.md      # Version history
```

File Index Contract

Every skill that creates or modifies files must include a file index - a declaration of exactly which files the skill touches. This makes skills auditable, reversible, and composable without side effects.

```
## File Index Contract
CREATES:
- reports/{date}-{topic}-analysis.md
- reports/{date}-{topic}-data.json

READS:
- backlog/ACTIVE-BACKLOG.md (read-only)
- config/settings.yaml (read-only)

MODIFIES:
- None

NEVER TOUCHES:
- .env (secrets)
- node_modules/
- .git/
```

5-Step Skill Creation Process

1. Define the trigger. What specific condition or command invokes this skill? Be precise enough that the skill doesn't activate unexpectedly.
2. Map the workflow. Write the steps as numbered actions. If a step is ambiguous, decompose it further.
3. Identify inputs and outputs. What does the skill receive? What does it produce? Define the schemas explicitly.
4. Write the edge cases. What are the 3 ways this skill most commonly fails? Add handling for each.
5. Write the tests. Create at least 3 test cases: happy path, edge case, failure case. The skill isn't complete without passing tests.

Quality Checklist

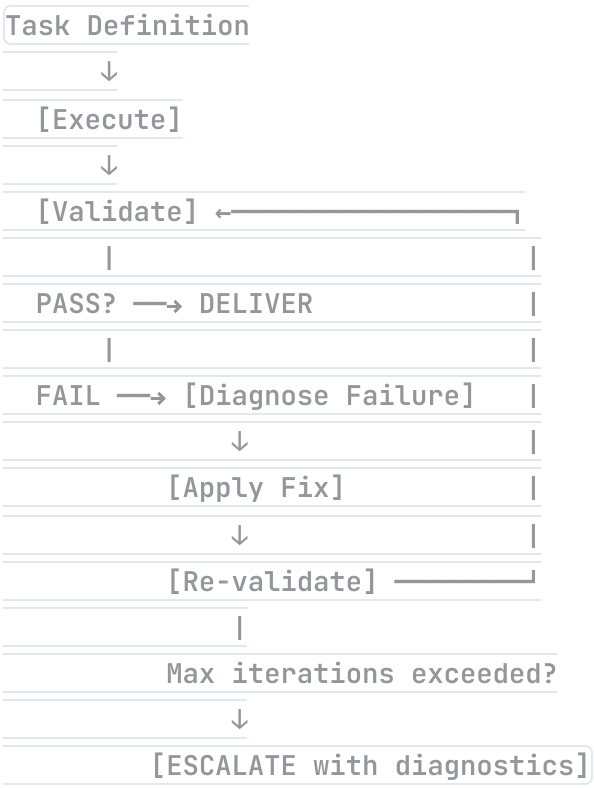
- ✓ YAML frontmatter is complete and valid
- ✓ Trigger condition is specific enough to avoid false activation
- ✓ All inputs and outputs are typed and documented
- ✓ Workflow has numbered steps with no ambiguous steps
- ✓ At least 2 concrete examples in the skill
- ✓ Error handling covers the 3 most likely failure modes
- ✓ File index contract is present (if skill touches filesystem)
- ✓ Tests exist and pass

CHAPTER 23

Closed-Loop Self-Correction

> *The closed-loop pattern transforms a single-pass AI into a self-correcting system. Instead of hoping the first output is good enough, the system validates, detects failures, corrects, and re-validates - automatically, without human intervention.*

The Closed-Loop Cycle



The critical design principle: validation must be concrete and binary. "Does this look good?" is not a validation. "Does this JSON parse with all required keys present?" is a validation. "Are all unit tests passing?" is a validation. Binary checks enable automatic failure detection without human judgment.

Builder/Validator Agent Pair

The strongest implementation of closed-loop correction separates builder and validator into independent agents. The builder builds. The validator validates - with no knowledge of what the builder was trying to do,

only what the output should look like. This separation prevents the builder from unconsciously validating its own work against its own assumptions.

```
BUILDER_PROMPT = """
AGENT: Builder
TASK: {task_specification}
BUILD the required artifact. Focus on correctness.
Output the artifact and nothing else.
"""

VALIDATOR_PROMPT = """
AGENT: Validator
ARTIFACT: {builder_output}
CRITERIA: {acceptance_criteria}

For each criterion, check the artifact and output:
PASS [criterion]  -  [evidence]
or
FAIL [criterion]  -  [specific failure]  -  [what fix is needed]

Final line must be: VERDICT: PASS or VERDICT: FAIL
"""

def closed_loop(task, criteria, max_iterations=3):
    artifact = None
    for i in range(max_iterations):
        artifact = builder(task, previous_failures=failures_so_far)
        validation = validator(artifact, criteria)
        if validation.verdict == "PASS":
            return artifact
        failures_so_far.append(validation.failures)
        task = task_with_fixes(task, validation.failures)

    escalate(task, artifact, failures_so_far)
```

CHAPTER 24

Consensus & Voting Patterns

> *When the stakes are high and ambiguity is real, multiple independent agents voting on the same question produces more reliable judgments than any single agent. This chapter covers the F-Thread pattern - the most practical implementation of AI consensus.*

The F-Thread Pattern

F-Thread (fan-out thread) sends the same question to N independent agents, then synthesizes their responses. The key: agents must not see each other's responses before forming their own. Independence is the mechanism - if agents can see each other, they'll anchor on the first response, destroying the statistical advantage of multiple independent perspectives.

When to Use Consensus

SITUATION	USE CONSENSUS?	WHY
High-stakes, irreversible decision	✅ Yes	Cost of error exceeds cost of multiple agents
Ambiguous problem with multiple valid framings	✅ Yes	Different agents catch different framings
Security review, compliance check	✅ Yes	Redundancy catches issues any single review misses
Routine, well-specified task	❌ No	Over-engineering; single agent is sufficient
Creative task with subjective quality	⚠️ Maybe	Useful for "best of N" but not true consensus

Complete F-Thread Template

```
# F-Thread with synthesis  -  production-ready template

QUESTION = """
[High-stakes decision or question requiring consensus]
"""

CONTEXT = """
[All relevant context  -  same for all agents]
"""

AGENT_PROMPT = """
You are one of 5 independent reviewers analyzing this question.
You do NOT know what other reviewers concluded.

QUESTION: {QUESTION}
CONTEXT: {CONTEXT}

Provide your INDEPENDENT analysis:

1. RECOMMENDATION: [clear position]
2. CONFIDENCE: [0-100%]
3. PRIMARY REASONS: [top 3, in order of importance]
4. STRONGEST COUNTERARGUMENT: [best argument against your position]
5. KEY UNCERTAINTY: [what you're least sure about]
6. DEAL-BREAKER: [what would make you reverse your recommendation]
"""

# Run 5 agents in parallel (truly independent)
responses = await asyncio.gather(*[
    agent(AGENT_PROMPT) for _ in range(5)
])

SYNTHESIS_PROMPT = """
You are synthesizing 5 independent expert analyses.

QUESTION: {QUESTION}
ANALYSES: {responses}

SYNTHESIS FRAMEWORK:
1. AGREEMENT LEVEL: Strong (4-5 agree) / Moderate (3 agree) / Split (2-3 agree)
2. CONSENSUS POSITION: What most agents recommend
3. DISSENTING VIEW: What minority agents recommend and why
4. SHARED CONCERNS: Issues ALL agents raised
5. UNIQUE INSIGHTS: Issues only 1-2 agents caught (often most valuable)
```

```
6. CONFIDENCE DISTRIBUTION: Average, range, and any outliers
7. FINAL RECOMMENDATION: Synthesized judgment
8. CONFIDENCE IN SYNTHESIS: [0-100%]
9. ESCALATE TO HUMAN IF: [conditions that require human judgment]
"""

final = synthesizer(SYNTHESIS_PROMPT)
```

PART VI

The 8 Development Patterns Habits of Effective AI Engineers

These eight patterns aren't specific to prompting - they're habits of effective engineering applied to AI systems. Practitioners who internalize these patterns ship better systems faster, with fewer production incidents and better recoverability when things go wrong.

CHAPTER 25

The 8 Development Patterns

> *A pattern is a named, reusable solution to a recurring problem. These eight solve the problems that most consistently cause AI engineering projects to fail - not due to AI limitations, but due to process failures.*

Pattern 1: Iron Law (Test First)

Principle: Write the test before writing the implementation. The test specifies what "done" means.

Why it matters: Without a test, "done" is undefined and expands infinitely. With a test, done is a binary state. This is even more important in AI systems than in traditional software, because AI output is probabilistic - without explicit success criteria, you're evaluating vibes, not correctness.

✕ NO TEST FIRST

```
Task: Build email classifier
[Build classifier]
"Looks pretty good to me. Ship it."
[Deploy]
[3 days later: 40% misclassification rate discovered]
```

✓ IRON LAW APPLIED

```
Task: Build email classifier
Test: must achieve >95% accuracy on test set
  of 500 labeled emails (provided)
[Build classifier]
[Run: pytest tests/classifier.py → FAIL: 87%]
[Iterate until PASS: >95%]
[Deploy with known accuracy baseline]
```

Apply to: All code changes. All new prompts. All new agents. Define "done" before you start.

Pattern 2: Reconnaissance (Look Before You Leap)

Principle: Before modifying anything, fully understand the current state. Read before writing. Map before building.

Why it matters: The most expensive engineering mistakes come from not understanding what already exists. In AI codebases especially, where agents can generate hundreds of files quickly, "building in isolation" is endemic. Reconnaissance prevents redundancy and prevents breaking things that worked.

```
# Reconnaissance protocol for any code modification
1. Read the file before editing it
2. Search for other files that import/use this code
3. Check git log for recent changes and their reasoning
4. Search for related tests before writing new ones
5. Check if a similar pattern already exists in the codebase
6. THEN make your change

# Reconnaissance prompt pattern
<task>
Before implementing the solution:
1. List all files relevant to this change
2. Describe what each file currently does
3. Identify what currently breaks if this change is made incorrectly
4. Only then: implement the solution
</task>
```

Pattern 3: One Question (Ask One Thing at a Time)

Principle: When you need to ask an AI or a human for input, ask one question at a time. Wait for the answer before asking the next.

Why it matters: Multi-question prompts produce multi-answer responses where each answer influences and potentially degrades the others. Asking one focused question produces one focused answer. The second question, asked with the context of the first answer, produces a better second answer than if both had been asked simultaneously.

× MULTI-QUESTION (DILUTED ANSWERS)

```
"What's the best database for this use case?
What schema would you recommend? What indexes
should I create? And how should I handle
migrations?"
```

✓ ONE QUESTION (FOCUSED ANSWERS)

```
Turn 1: "What's the best database for this use case?"
[Get answer: PostgreSQL, with reasoning]

Turn 2: "Given PostgreSQL, what schema?"
[Get schema based on actual DB recommendation]

Turn 3: "Given this schema, what indexes?"
[Get indexes tuned to the actual schema]
```

Pattern 4: Incremental Validation

Principle: Confirm that each step works before proceeding to the next.

Why it matters: Error propagation is exponential in multi-step systems. A wrong assumption in step 2 makes step 3 wrong, which makes step 4 doubly wrong. By confirming at each step, you eliminate entire classes of cascading errors.

```
<incremental_validation_protocol>
For each step in this multi-step task:

1. Complete the step
2. State what you've done and what the output is
3. State whether you're confident the step is correct
4. If not confident: ask for clarification before continuing
5. Wait for confirmation to proceed

DO NOT CONTINUE to the next step until the current step
is explicitly confirmed.
</incremental_validation_protocol>
```

Pattern 5: Black-Box Scripts (Use --help, Not Source)

Principle: Use tools as documented black boxes. Read the --help output, not the source code, to understand what a tool does.

Why it matters: Reading source code to understand what a CLI does is slower and error-prone. The documented interface (--help, man pages, README) is the contract. Using it as intended reduces unexpected behavior and respects the tool's design intent.

```
# Wrong: reverse-engineering a tool from source
cat /usr/local/bin/deployment-tool | grep "function " | ...

# Right: use the documented interface
deployment-tool --help
deployment-tool deploy --help
deployment-tool deploy --dry-run --env staging
```

Applied to prompting: When integrating with an API or SDK, read the documentation and examples first. Don't infer behavior from implementation details you can read - use the documented interface.

Pattern 6: Progressive Fallback

Principle: Try the best approach first. If it fails, fall back to a simpler approach. Have a degraded-but-functional minimum viable path.

Why it matters: Production systems that fail completely when their primary path is unavailable are fragile. Systems that degrade gracefully survive. In AI pipelines, this means: if the primary model fails, fall back to a secondary model. If the complex extraction fails, fall back to a simpler heuristic. Never let perfect be the enemy of functional.

```
async def robust_extract(text):
    # Level 1: Best - structured extraction with Claude
    try:
        result = await claude_extract(text, model="sonnet")
        if validate(result): return result
    except: pass

    # Level 2: Fallback - cheaper model
    try:
        result = await claude_extract(text, model="haiku")
        if validate(result): return result
    except: pass

    # Level 3: Minimum viable - regex heuristics
    result = regex_fallback(text)
    return result or {"status": "extraction_failed", "raw": text}
```

Pattern 7: Simplicity Criterion

Principle: The simplest solution that meets the requirements is the best solution. Deletions that maintain quality are celebrated, not suspicious.

Why it matters: Complexity compounds. Every unnecessary line of code, every unnecessary agent, every unnecessary prompt component is a future bug surface. The instinct to add more (more instructions, more agents, more context) is almost always wrong. The discipline to remove is a professional superpower.

> *"When in doubt about whether to add something, remove it instead. If removing it breaks something important, add it back with a comment explaining why it's necessary. If nothing breaks, congratulations - you just improved your system."*

Applied to prompts: After writing a prompt, read every sentence and ask: "Would the output be meaningfully worse without this?" If the answer is no, delete it. Ideal prompts are minimal specifications, not comprehensive manuals.

Pattern 8: Knowledge Delta

Principle: Before adding something to your system, check if it already exists. Only add what's genuinely new - the "delta" from the current state of knowledge.

Why it matters: AI systems, especially agentic ones that generate their own artifacts, are prone to duplication. The same functionality implemented in three different places, with slight variations, creates a maintenance nightmare. Knowledge delta thinking forces you to inventory before you build.

```
<knowledge_delta_check>
Before implementing this:
1. Search the codebase for similar functionality
2. Search the skill library for existing skills that overlap
3. Check if the pattern is documented in CLAUDE.md
4. Check if a library/package already does this

Only implement what doesn't already exist.
If something similar exists: extend it, don't duplicate it.
Report: "Found [X], extending it" or "Nothing similar found, building new."
</knowledge_delta_check>
```

PART VII

The Deadly Seven Anti-Patterns What to Stop Doing Immediately

These seven patterns appear in 80% of underperforming AI systems. They're common because they feel natural or seem like best practices. They're deadly because they produce systematically bad outputs while making it hard to diagnose why.

CHAPTER 26

The Deadly Seven Anti-Patterns

> *An anti-pattern is a seemingly reasonable approach that consistently produces bad outcomes. These seven are the most prevalent, most damaging, and most easily fixed once you recognize them.*

Anti-Pattern 1: Wall of Text

What it looks like: A single paragraph containing instructions, context, examples, and the task, all run together with no structure.

Why it fails: Attention dilution. When everything has equal visual weight, the model allocates roughly equal attention to everything - including the irrelevant. Critical constraints are missed. Format requirements buried in the middle are ignored. The model does its best to extract intent from an undifferentiated mass of text.

✕ WALL OF TEXT

```
"Please help me analyze this customer feedback data
which I've collected over the past quarter and I need
you to identify the key themes and also look for any
negative sentiment that might be urgent and please
format it nicely and make sure to include a summary
at the top and maybe some statistics too and try to
keep it under 500 words but also be comprehensive..."
```

✓ STRUCTURED

```
## TASK
Analyze Q3 customer feedback for key themes
and urgent negative sentiment.

## OUTPUT FORMAT
1. Executive summary (100 words max)
2. Top 5 themes (name + frequency + example)
3. Urgent issues (sentiment score < -0.7)
4. Statistics: volume, avg sentiment, breakdown
```

```
## CONSTRAINTS
- Under 500 words total
- Flag anything requiring immediate action
```

Anti-Pattern 2: Vague Success

What it looks like: Instructions that don't define what a successful output looks like. "Make it good." "Be thorough." "Write something professional."

Why it fails: Without a definition of success, the model optimizes for what's most probable given the training data - which may be very different from what you need. "Professional" means different things in different industries, companies, and contexts. Define it.

```
# Anti-pattern: vague success
"Write a professional email about the project delay."

# Fixed: defined success
"Write an email to an enterprise client (VP-level)
about a 2-week project delay.
SUCCESS CRITERIA:
- States the delay duration on line 1
- Gives one concrete reason (not multiple excuses)
- States the new delivery date
- Includes one specific action we're taking
- Does NOT apologize more than once
- Does NOT use passive voice
- Under 200 words"
```

Anti-Pattern 3: Context Dumping

What it looks like: Pasting entire files, entire databases, entire conversation histories - "more context can't hurt."

Why it fails: It hurts in three ways: (1) attention dilution - relevant signal drowns in noise; (2) context window waste - tokens spent on irrelevant content are tokens unavailable for reasoning; (3) confusion - models have trouble identifying what's relevant when everything is presented as equally important.

How to fix: Apply the 30-70% smart zone rule. Extract and include only what's directly relevant to the current task. If you're debugging a function, include that function and the functions it calls - not the entire codebase. If you're analyzing a dataset, include the relevant columns and a sample - not every row.

Anti-Pattern 4: Missing Examples

What it looks like: Instructions that describe a complex output format without showing what it looks like.

Why it fails: Descriptions of desired output are systematically less informative than examples of desired output. "Return a JSON object with nested arrays representing the hierarchy" is ambiguous. Showing the exact JSON schema with realistic values eliminates ambiguity completely. Anthropic's research shows examples are the second most impactful technique - not using them wastes enormous potential.

```
# Anti-pattern: description without example
"Return a hierarchical JSON structure with the
department tree, including employee counts and
budget rollups at each level."

# Fixed: include concrete example
"Return this exact JSON structure:
{
  'department': 'Engineering',
  'headcount': 45,
  'budget': 2400000,
  'children': [
    {
      'department': 'Frontend',
      'headcount': 12,
      'budget': 640000,
      'children': []
    }
  ]
}"
```

Anti-Pattern 5: No Self-Check

What it looks like: Single-pass generation with no validation step. The model generates output and it's used as-is.

Why it fails: Language models make mistakes - not randomly, but systematically. They overclaim certainty, miss edge cases, produce slightly wrong numbers, and drift from format requirements. A self-check step catches these systematic failures before they become downstream problems.

How to fix: Add a self-evaluation block to every production prompt (see Chapter 16). At minimum: "Before returning your response, verify that [criterion 1], [criterion 2], and [criterion 3] are met."

Anti-Pattern 6: Monolith Prompt

What it looks like: One enormous prompt attempting to research + analyze + draft + review + format - all in one shot.

Why it fails: Monolith prompts fail in three ways: (1) attention dilution across multiple competing tasks; (2) error propagation - a wrong turn in "research" contaminates "analysis" which contaminates "draft"; (3) undebuggability - when the output is wrong, you can't tell which step failed.

How to fix: Decompose into a prompt chain (Chapter 10). Each step does one thing. The output of each step is inspectable and correctable before proceeding. The whole chain produces better results than any monolith, even if each individual step is simpler.

Anti-Pattern 7: Adjectives Over Structure

What it looks like: Prompts heavy on quality adjectives ("excellent," "comprehensive," "detailed," "insightful") and light on structural specification.

Why it fails: Adjectives are not constraints. "Excellent" is not a measurable criterion. "Comprehensive" doesn't tell the model what to include. Adjective-heavy prompts produce outputs that feel like they're trying to be excellent - verbose, hedge-everything, over-qualified - rather than actually meeting a defined bar.

✕ ADJECTIVES OVER STRUCTURE

```
"Write an excellent, comprehensive, insightful,
and highly detailed analysis that thoroughly
covers all important aspects with great clarity
and professional expertise."
```

✓ STRUCTURE OVER ADJECTIVES

```
"Write a competitive analysis covering:
1. Market position (1 paragraph, cite data)
2. Feature comparison vs top 3 competitors (table)
3. Pricing analysis (table with percentages)
4. Identified gaps and opportunities (bullet list)
Length: 600-800 words. No intro/conclusion paragraphs."
```

Signs Your Prompt Needs Restructuring

⚠️ The prompt is a single long paragraph with no structure

⚠️ Success criteria are adjectives, not measurable specifications

⚠️ No examples of the desired output format are provided

⚠️ Context exceeds 70% of the context window

⚠️ The prompt asks for more than 3 distinct things at once

⚠️ No self-validation step at the end

⚠️ Output quality varies significantly between runs

⚠️ You've added to the prompt 5+ times without removing anything

PART VIII

Applied Domains Templates for Real Work

The principles and patterns from previous parts applied to the domains where most practitioners spend most of their time. Each chapter provides 10 production-ready templates you can use immediately.

CHAPTER 27

Software Engineering Prompts

> *Software engineering is the domain where prompt engineering pays off the most. Code is verifiable, which means you can apply the Iron Law strictly and measure quality precisely. These templates have been battle-tested in production codebases.*

Code Generation Best Practices

The most common failure in code generation prompts is ambiguity about the environment. "Write a function to sort users by date" fails because: which language? Which date format? Which sort order? What does the User type look like? Is it in-place or returning a new array? These ambiguities produce code that looks right but doesn't integrate. Specify the environment completely.

10 Production-Ready Templates

1. Implement Feature

```
## CONTEXT
Language: TypeScript 5.3 (strict)
Framework: Express 4.x
Database: PostgreSQL 16 via Prisma
Auth: JWT (existing middleware at src/middleware/auth.ts)
Error handling: uses AppError class (src/lib/errors.ts)

## TASK
Implement: POST /api/users/:id/preferences
Accepts: { theme: 'light'|'dark', notifications: boolean }
Returns: updated user object
Requirements:
- Authenticated route (use existing auth middleware)
- Validate input with Zod
- Update via Prisma (model: User, fields: theme, notificationsEnabled)
- Return 200 + user on success, 422 + validation errors on invalid input
- 404 if user not found
```

```
## OUTPUT
1. Route handler (src/routes/users.ts addition)
2. Zod schema (can inline or separate)
3. Integration test (tests/routes/users.test.ts pattern)
No explanation needed  -  code + tests only.
```

2. Code Review

```
## ROLE
Senior engineer reviewing for production readiness.
Optimize for: correctness, security, maintainability.

## REVIEW SCOPE
[paste diff or files]

## CRITERIA
BLOCKERS (must fix before merge):
- Logic errors or off-by-one issues
- Security vulnerabilities (injection, auth bypass, data exposure)
- Missing error handling for realistic failure modes
- Breaking changes to public interfaces without version bump

MAJORS (should fix this PR):
- Performance issues (N+1 queries, unnecessary loops)
- Missing tests for critical paths
- Violated conventions in this codebase

MINORS (backlog acceptable):
- Style inconsistencies
- Naming that could be clearer
- Documentation gaps

## OUTPUT FORMAT
For each issue: File + line + severity + description + suggested fix.
End with: APPROVE | REQUEST_CHANGES | BLOCKED
```

3. Debug This

```
## ERROR
[paste exact error message and stack trace]

## CONTEXT
[paste the failing code + any directly related code]

## ENVIRONMENT
[language version, framework, relevant dependencies]

## WHAT I'VE TRIED
[list attempts, even if they seemed unrelated]

## TASK
1. Identify the root cause (not the symptom)
2. Explain WHY this causes the observed error
3. Provide the minimal fix
4. Identify if this same bug pattern exists elsewhere

Constraints: minimal fix only. No refactoring.
```

4. Write Tests

```
## TARGET
[paste the function/module to test]

## FRAMEWORK
[Vitest / Jest / Pytest / etc. + testing utilities available]

## COVERAGE REQUIREMENTS
- Happy path (all valid inputs)
- Edge cases: null, undefined, empty string, zero, negative numbers
- Error cases: what exceptions should be thrown and when
- Boundary values: min, max, min-1, max+1 where applicable

## OUTPUT
Complete test file using describe/it blocks.
Test descriptions must read as specifications:
"should return X when Y" not "test 1".
```

5. Refactor for Readability

```
## CODE TO REFACTOR
[paste code]

## GOALS
- Extract functions over 20 lines into named helpers
- Replace magic numbers/strings with named constants
- Clarify variable names (no abbreviations)
- Add JSDoc to public functions

## CONSTRAINTS
- Zero functional changes. Same inputs → same outputs.
- Do not change public interface signatures
- Do not add features
- Maintain existing test coverage (tests must still pass)

Confirm before outputting: state the 3 main changes you'll make.
```

6. Performance Optimization

```
## CONTEXT
This function runs on every HTTP request (high frequency).
Current p99 latency: [X ms]. Target: [Y ms].
Profile shows: [hotspot location if known]

## CODE
[paste function or module]

## ANALYSIS REQUIRED
1. Identify all O(n²) or worse operations
2. Identify N+1 query patterns
3. Identify unnecessary recomputation
4. Recommend optimizations in order of expected impact
5. For each: estimated complexity improvement + risk of change
```

7. Database Query Optimization

```
## QUERY
[paste slow query + EXPLAIN ANALYZE output if available]

## SCHEMA
[paste relevant table definitions with current indexes]

## CONTEXT
Table sizes: [table_a: 5M rows, table_b: 100K rows]
Query frequency: [X per second]
Current execution time: [X ms]

## TASK
1. Diagnose why query is slow
2. Propose optimized query + index changes
3. Estimate improvement (order of magnitude is fine)
4. Flag any data integrity risks from index changes
```

8. Security Review

```
## CODE
[paste authentication / authorization / input handling code]

## CHECK FOR
- SQL/NoSQL injection (parameterized queries?)
- XSS (user input ever rendered as HTML?)
- Auth bypass (can non-admin reach admin routes?)
- Timing attacks (constant-time comparison for secrets?)
- Session management (token entropy, expiry, rotation)
- Rate limiting (is this endpoint protected?)
- Sensitive data in logs (passwords, tokens, PII?)

## OUTPUT
Issues: severity + exact location + exploit scenario + fix.
OWASP category for each issue.
```

9. Architecture Design

```
## PROBLEM STATEMENT
[What we're building, at what scale, with what constraints]

## CONSTRAINTS
- Team size: [N engineers]
- Timeline: [X weeks to first production]
- Scale targets: [N requests/sec, N users, N GB data]
- Tech stack preferences/mandates: [any non-negotiables]
- Budget constraints: [if relevant]

## TASK
Propose 2 architectures (not 3  -  two forces a real tradeoff):
For each:
- Core components and their responsibilities
- Data flow diagram (text-based is fine)
- Where this design excels
- Where this design struggles
- Recommended team structure

Then: your recommendation and the single deciding factor.
```

10. Migration Script

```
## CURRENT STATE
[Data structure / schema before migration]

## TARGET STATE
[Data structure / schema after migration]

## REQUIREMENTS
- Idempotent: can run multiple times safely
- Rollback: includes rollback procedure
- Batched: processes in batches of [N] to avoid lock contention
- Progress: logs progress every [N] records
- Dry run: --dry-run flag shows what would change without changing it

## CONSTRAINTS
```

```
- Zero downtime (data must remain accessible during migration)
- Production has [N] records  -  estimate runtime at batch size [N]
```

CHAPTER 28

Content & Writing

> *AI-assisted writing fails in predictable ways: generic voice, structural mediocrity, and content that reads as obviously AI-generated. These templates are designed to avoid those failures by being specific about voice, structure, and quality criteria.*

The Research → Outline → Draft → Polish Workflow

Never generate a final piece in one shot. The four-stage workflow produces dramatically better output: research (gather evidence and arguments), outline (structure the argument before committing to words), draft (write fast, fill the structure, don't edit), polish (cut and sharpen - add nothing, remove everything unnecessary). Each stage uses a separate prompt.

Voice Preservation

The most common complaint about AI writing: "It doesn't sound like me." The fix: provide writing samples and a voice description, not just topic instructions. A model that has seen how you actually write can match your patterns, not impose its own.

```
## VOICE REFERENCE
Here are 3 examples of my writing. Study the voice:

[Example 1 - 2-3 paragraphs of your actual writing]
[Example 2]
[Example 3]

## VOICE CHARACTERISTICS (extracted from my samples)
- Short sentences. Average 12 words.
- No passive voice.
- First word of each paragraph is never "The" or "I"
- Technical terms used without apology - no definition padding
- Dry humor occasionally - never filler enthusiasm
- Concrete examples before abstractions

## TASK
Write [piece] in this voice.
```

```
After writing, check: does this sound like the author above?
If it sounds generic, rewrite until it doesn't.
```

10 Writing Templates

1. Blog Post (Technical)

```
Write a technical blog post for [audience] about [topic].
Structure:
- Hook: a specific problem or counterintuitive claim (no intro)
- Section 1: [claim 1] + evidence/example
- Section 2: [claim 2] + evidence/example
- Section 3: [claim 3] + evidence/example
- Close: the one thing to remember, and the next step

Length: [700-900] words. No fluff paragraphs.
Constraint: every paragraph must earn its place  -  if it
could be cut without weakening the argument, cut it.
```

2. Hook Improvement

```
Current opening:
[paste current intro/hook]

Problems to fix:
- If it starts with "In today's world..." → rewrite completely
- If it takes more than 2 sentences to state the topic → cut
- If it uses passive voice → rewrite active
- If it makes no specific claim → add one

Write 3 alternative hooks that:
1. State a specific, debatable claim
2. Use an unexpected comparison or data point
3. Open mid-story (in medias res)

Constraints: each under 40 words. No fluff.
```

3. Email → Concise

```
Rewrite this email. Cut word count by 40%.
Rules:
- Lead with the ask or the key information, not background
- One sentence per idea (no compound sentences with "and")
- Cut every word that doesn't change meaning without it
- Preserve: all names, dates, numbers, specific asks
- Keep exactly one call to action (the most important)

Original: [paste]
Target: [word count target]
```

4. Executive Summary

```
Write a 150-word executive summary of this [report/doc].
[paste document]

Required elements (in this order):
1. What was done / what this is (1 sentence)
2. The single most important finding (1-2 sentences)
3. The financial or strategic implication (1 sentence)
4. The recommended action (1 sentence)

Audience: C-suite with 90 seconds to read.
Voice: declarative statements, no hedging, no jargon.
Constraint: 150 words exactly (±5 words).
```

5. Thread / LinkedIn Post

```
Convert this [article/idea] into a LinkedIn post.

FORMAT:
- Line 1: specific claim or surprising insight (hook)
- Lines 2-4: 3 short supporting points (one per line)
- Lines 5-7: the implication or takeaway
- Final line: question or CTA that invites response
```

```
CONSTRAINTS:
- No emojis (professional audience)
- No "I'm excited to share" type openers
- No bullet points  -  short paragraphs only
- Under 200 words
- Must make one specific claim, not vague inspiration

Source material: [paste]
```

6. Research Brief

```
Research [topic] and produce a structured brief.

REQUIRED SECTIONS:
1. Core question: What is the actual question this research addresses?
2. Current consensus: What do most experts agree on?
3. Active debates: Where do experts disagree, and why?
4. Key data: 3-5 statistics (cite source + date for each)
5. Implications: For [specific context/industry/decision]
6. Knowledge gaps: What's not yet known?
7. Sources: List all sources used

CONSTRAINTS:
- Distinguish clearly: established fact vs. expert opinion vs. your synthesis
- Flag when you're uncertain
- No sources older than 2023 without noting their age
```

7. Presentation Script

```
Write a [15-minute] presentation script on [topic] for [audience].

STRUCTURE:
- Opening: surprising question or data point (no "Today I'll talk about")
- Agenda: 3 main points in one sentence each
- Point 1: [claim] + evidence + so what
- Point 2: [claim] + evidence + so what
- Point 3: [claim] + evidence + so what
- Close: single takeaway + call to action
```

```
SPEAKER NOTES: Include timing per section.
SLIDE MARKERS: Note where slides should change [SLIDE].
VOICE: conversational, not written-to-be-read.
AVOID: "As you can see," "Let's take a look at," "In conclusion"
```

8. Argument Steelman

```
I believe: [your position]

TASK: Construct the strongest possible argument AGAINST
my position. This is for debate preparation.

Requirements:
- Find the 3 strongest counterarguments (not straw men)
- Use the best available evidence for each
- Identify the weakest point in my argument
- Describe a realistic scenario where I'm wrong

Then: having steelmanned the opposition, where does my
position genuinely need to be qualified or nuanced?
What should I concede to make my position more defensible?
```

9. Critique & Improve

```
Critique this writing. Be specific  -  no vague feedback.

[paste writing]

CRITERIA:
- Clarity: Is every sentence instantly understandable?
- Structure: Does each paragraph have a single clear point?
- Evidence: Are claims supported with specific evidence?
- Voice: Is the tone consistent throughout?
- Concision: What can be cut without loss?

For each issue: quote the exact problem text + rewrite it.
Priority: show the 3 highest-impact improvements first.
```

10. Ghostwrite to Voice

```
## MY VOICE (from samples above)
[voice description extracted from samples]

## IDEAS TO COVER
[bullet points of the key ideas, in any order]

## TASK
Write [type: essay / post / thread] covering these ideas
in my voice. Organize for impact, not the order I listed them.

QUALITY BAR: If someone who knows me read this, they should
not be able to tell whether I wrote it or an AI did.
If it reads as generically AI-written, rewrite it.
```

CHAPTER 29

Image Generation

> *Image generation prompts are the most specific domain in this guide - successful prompts require combining subject description, lighting architecture, camera specification, and style reference with precision. Vague image prompts produce generic images. Specific prompts produce professional-grade output.*

The Four Pillars of Image Prompts

PILLAR	WHAT TO SPECIFY	EXAMPLE
Subject	Who/what, doing what, in what context	"software engineer at standing desk in modern office"
Lighting	Quality, direction, color temperature	"soft window light from left, warm 4000K, gentle shadows"
Camera	Lens focal length, aperture, angle	"85mm portrait lens, f/1.8 bokeh, eye-level"
Style	Genre, era, film stock, post-processing	"editorial photography, Kodak Portra 400 film grain"

Camera & Lens Specifications

```
# Portrait photography
85mm f/1.4, shallow depth of field, bokeh background,
subject sharp, eye-level angle

# Architecture / wide interior
24mm wide angle, f/8 deep focus, straight horizons,
geometric symmetry, slight perspective correction

# Product photography
```

```
100mm macro, f/11 full depth of field,
45-degree angle, clean white or gradient background

# Street photography
35mm, f/5.6, zone focusing, handheld look,
slightly raised angle, environmental context visible

# Cinematic landscape
Anamorphic lens look, 2.39:1 aspect ratio,
lens flare, golden hour directional light
```

Lighting Architecture

```
# Studio portrait lighting
"3-point studio lighting: key light 45° right,
fill light 60% power left, hair light above,
white backdrop, even illumination, commercial photography"

# Natural window light
"large north-facing window light, overcast sky (soft diffused),
no hard shadows, desaturated natural tones, lifestyle photography"

# Dramatic cinematic
"single practical source (desk lamp), noir shadows,
high contrast, 80% shadows, moody atmosphere,
neo-noir color grade"

# Golden hour outdoors
"magic hour directional sunlight, low angle, warm orange tones,
long shadows, lens flare, rich golden saturation"
```

10 Image Generation Templates

```
# 1. Professional Headshot
"Professional headshot of [description], natural window light
from left, 85mm portrait lens, shallow depth of field,
soft bokeh background (blurred modern office), eye-level,
warm skin tones, slight smile, business casual attire,
```

clean and approachable, LinkedIn style, photorealistic"

 # 2. Product on Clean Background
"[Product] on pure white seamless background, 3-point studio lighting, 100mm macro lens, f/16 full sharpness, subtle drop shadow below product, commercial product photography, highly detailed, 8K render quality"

 # 3. Cinematic Scene
"[Scene description], cinematic photography, anamorphic lens flare, teal and orange color grade, 2.39:1 aspect, film grain, dramatic lighting, hyper-detailed, directed by Roger Deakins, golden hour, atmospheric haze"

 # 4. UI/App Screenshot
"Clean app interface screenshot, [app description], iOS design language, SF Pro typeface, light mode, precise pixel-perfect design, modern flat design, professional mockup on iPhone 16 Pro, white background"

 # 5. Architectural Render
"Architectural visualization of [building], photorealistic, 8K render, golden hour lighting, professional 3D rendering, Lumion or Unreal Engine quality, people for scale, landscaping, atmospheric sky, accurate materials and textures"

 # 6. Concept Art (Game/Film)
"Concept art for [scene/character/environment], highly detailed, professional concept art, ArtStation style, dynamic composition, cinematic lighting, rich color palette, Feng Zhu influence, detailed brush work"

 # 7. Infographic / Data Visualization
"Clean infographic about [topic], modern flat design, [color palette], clear hierarchy, icons and charts, professional layout, sans-serif typography, white background, Figma/Canva quality design"

 # 8. Documentary / Editorial Portrait
"Documentary portrait of [subject description], natural ambient light, slightly underexposed, journalistic style, 35mm film grain, desaturated, high contrast, environmental context visible in background, candidly posed"

 # 9. Abstract Brand Visual

```
"Abstract brand visual for [company/concept], modern
geometric forms, [brand colors], dynamic composition,
professional graphic design, poster quality, minimalist
but rich, suitable for marketing materials"

# 10. Technical Diagram
"Technical architecture diagram, clean vector style,
[system description], professional diagramming tool aesthetic,
clear nodes and connections, labeled, color-coded by type,
white background, Lucidchart or Draw.io quality"
```

CHAPTER 30

Business & Strategy

> *Strategic analysis prompts fail when they're too open-ended to produce actionable output. The templates in this chapter apply the specificity and structure principles from earlier chapters to the messy, ambiguous problems of business strategy.*

10 Business Templates

1. Competitive Analysis

```
Analyze the competitive landscape for [company/product].

STRUCTURE:
1. Market definition: What is the actual market? (Beware
   of defining it too broadly or too narrowly)
2. Top 5 competitors: Name, positioning, primary strength,
   primary weakness
3. Feature comparison matrix: [table with key dimensions]
4. Pricing analysis: price points, model, any pricing power signals
5. Competitive moats: What would make these positions defensible?
6. Whitespace: What is no competitor currently doing well?
7. Verdict: Where is competitive position weakest? Where is it strongest?

CONSTRAINT: Separate what's publicly documented from what's inferred.
Label each with [FACT] or [INFERENCE].
```

2. Business Model Canvas

```
Complete a Business Model Canvas for [company/idea].

NINE BLOCKS:
1. Customer Segments: Who are we creating value for?
2. Value Propositions: What value do we deliver? Which problem solved?
```

```
3. Channels: How do we reach segments? (awareness, eval, purchase, delivery, post-sale)
4. Customer Relationships: What type of relationship does each segment expect?
5. Revenue Streams: How does each segment pay? What are they willing to pay for?
6. Key Resources: What assets does the value proposition require?
7. Key Activities: What activities does the value proposition require?
8. Key Partnerships: Who are our key suppliers/partners? What do we get from each?
9. Cost Structure: What are the most important costs? Which resources/activities are most expensive?

For each block: 3-5 specific bullet points. No vague statements.
End with: The most critical assumption in this model that needs validation first.
```

3. Market Sizing

```
Estimate the market size for [product/service].

USE BOTH APPROACHES:
TOP-DOWN:
- Start with total addressable market (TAM) - cite source and year
- Apply segmentation filters to reach SAM (serviceable addressable market)
- Apply realistic penetration assumption to reach SOM (serviceable obtainable market)

BOTTOM-UP:
- Estimate number of target customers (show your math)
- Estimate annual revenue per customer (show your math)
- Calculate: customers × ACV = market opportunity

RECONCILE: Do top-down and bottom-up arrive at similar numbers?
If not - which is more reliable for this market, and why?

STATE ALL ASSUMPTIONS explicitly. Tag each as [conservative] [base] [optimistic].
```

4. SWOT with Strategy Matrix

```
SWOT analysis for [company/initiative], then strategic matrix.

SWOT:
```

```
- Strengths (internal, positive): 5 specific items
- Weaknesses (internal, negative): 5 specific items
- Opportunities (external, positive): 5 specific items
- Threats (external, negative): 5 specific items

STRATEGY MATRIX (the useful part most SWOTs skip):
- SO strategies: Use strengths to capture opportunities
- ST strategies: Use strengths to mitigate threats
- WO strategies: Address weaknesses to capture opportunities
- WT strategies: Defend against threats while addressing weaknesses

PRIORITY: Which single strategy has the best effort-to-impact ratio?
```

5. Unit Economics Analysis

```
Analyze unit economics for [business].

REQUIRED METRICS:
- CAC (Customer Acquisition Cost): total sales+marketing / new customers
- LTV (Lifetime Value): ACV × gross margin × (1/churn rate)
- LTV:CAC ratio (healthy = >3:1)
- CAC payback period: CAC / (ACV × gross margin / 12)
- Gross margin %
- Net revenue retention %
- Payback period

Given these numbers: [paste or describe available data]

ANALYSIS:
1. Which metrics are healthy vs concerning?
2. What's the primary driver of LTV:CAC?
3. What lever would most improve unit economics?
4. At what scale do unit economics become attractive for investors?
```

6. Go-to-Market Strategy

```
Develop a go-to-market strategy for [product].

CONTEXT:
```

```
- Product: [description]
- Target customer: [ICP  -  specific, not generic]
- Price point: [range]
- Current stage: [pre-launch / early traction / scaling]
- Team capacity: [sales, marketing resources]

GTM STRATEGY DOCUMENT:
1. ICP definition: firmographics + psychographics + trigger events
2. Positioning: one-sentence value proposition
3. Channel strategy: ranked by expected CAC and volume
4. Sales motion: self-serve / inside sales / enterprise (with rationale)
5. Land and expand: how do we grow revenue within accounts?
6. First 90 days: specific actions with owners and success metrics

CONSTRAINT: Be specific enough that a new hire could execute this.
```

7. Decision Analysis

```
Help me think through this decision:
[Describe the decision and options]

FRAMEWORK:
1. Clarify the actual decision: What is the real choice?
   (Often the stated choice is a symptom of an earlier undecided question)
2. Decision criteria: What matters? Weight each criterion 1-10.
3. Options analysis: Score each option against each criterion.
4. Risk analysis: What's the worst realistic outcome for each option?
5. Reversibility: How reversible is each option?
6. Recommendation: Given the criteria and weights, which option scores highest?

ANTI-SYCOPHANCY CONSTRAINT: If my framing of the options has
a better unstated alternative, name it before analyzing my options.
```

8. Investor Narrative

```
Help me construct the investor narrative for [company].

NARRATIVE ARC (10-slide structure):
1. The problem: Why does this matter? Who's in pain?
```

```
2. Solution: What do we do? (one sentence)
3. Why now: What changed that makes this possible/necessary?
4. Market size: TAM/SAM/SOM (realistic, cited)
5. Product: How does it work? Key differentiation.
6. Traction: What's happened so far? (metrics, not adjectives)
7. Business model: How do we make money? Unit economics.
8. Go-to-market: How do we acquire customers at scale?
9. Team: Why are we the right team for this?
10. Ask: How much, for what milestones, to prove what?

For each slide: draft 3 key points (not full slides).
Flag weak sections that need more work before investor meetings.
```

9. Operational Audit

```
Conduct an operational audit of [process/department].

AUDIT SCOPE: [describe what we're auditing]

CURRENT STATE ANALYSIS:
- What steps exist in the current process? (map it)
- Where are the bottlenecks? (time, cost, error rate)
- Where is manual work that could be automated?
- Where do handoffs fail? (information lost, delays added)

BENCHMARKS: Compare against industry standards where available.

RECOMMENDATIONS (prioritized by ROI):
For each:
- What to change
- Expected improvement (quantified)
- Implementation effort (low/medium/high)
- Dependencies and risks

QUICK WINS: What can improve in < 2 weeks with minimal risk?
```

10. Risk Register

```
Build a risk register for [project/initiative].

RISK CATEGORIES: Strategic, Operational, Financial, Technical,
Regulatory/Legal, People, External/Market

FOR EACH RISK:
- Risk description: specific, not vague
- Likelihood: Low (< 20%) / Medium (20-60%) / High (> 60%)
- Impact if realized: Low / Medium / High / Critical
- Risk score: Likelihood × Impact (1-9 scale)
- Early warning indicators: what would signal this risk is materializing?
- Mitigation: specific action to reduce likelihood
- Contingency: what we do if it happens anyway
- Owner: who monitors this risk

Prioritize: top 5 risks by risk score. For those 5: detailed mitigation plans.
```

APPENDIX A

Quick Reference Cards

The Core Four at a Glance

LEVER	OPTIMIZE BY	RED FLAG
Context	Minimize to essential; 30-70% smart zone	Dumping everything "just in case"
Model	Match tier to task complexity	Opus for Haiku tasks
Prompt	Structure > length	Paragraphs where bullets work
Tools	Fewer focused tools > many generic	50-tool agents

Template Selection Guide

USE THIS SCHEMA	WHEN YOU HAVE
Skill Definition	A reusable workflow called repeatedly by humans or agents
Slash Command	A lightweight operation triggered by a keyword (<200 tokens)
Agent System Prompt	A specialized subagent in a multi-agent pipeline
Four-Block Prompt	A structured one-shot request (INSTRUCTIONS/CONTEXT/TASK/FORMAT)
ADW Workflow	An end-to-end automation spanning multiple agents/tools
Mental Model YAML	Domain knowledge to load into agent context
Closed-Loop	A task requiring validation and self-correction

USE THIS SCHEMA	WHEN YOU HAVE
F-Thread Consensus	A high-stakes decision requiring independent verification

Persuasion Principles Quick Reference

PRINCIPLE	USE	AVOID
Authority	High-stakes technical tasks	Inflating stakes artificially
Commitment	Multi-step tasks; scope creep risk	High-ambiguity early decisions
Scarcity	Genuinely irreversible operations	Creative or synthesis tasks
Social Proof	Tasks with known failure patterns	Novel problems without precedent
Unity	Complex collaborative analysis	Simple instruction-following tasks
Reciprocity	Always - provide rich context	N/A - almost always beneficial
Liking	AVOID - creates sycophancy. Do not use.	

Anti-Patterns Checklist

- ✓ NOT a wall of text - has clear structure
- ✓ Success defined in measurable terms, not adjectives
- ✓ Context is minimal, not dumped
- ✓ Examples provided for non-trivial output formats
- ✓ Self-check or validation step included
- ✓ Complex tasks are chained, not monolithic

✓ **Structure specified, not just quality adjectives**

APPENDIX B

Model Comparison Guide

When Models Differ

The techniques in this guide work across Claude, GPT-4, and Gemini - but with different emphasis. Here are the most important model-specific optimizations.

TECHNIQUE	CLAUDE	GPT-4/O1	GEMINI
XML tags	✅ Highly effective (trained on XML)	⚠️ Moderate (use Markdown headers instead)	⚠️ Moderate
System prompts	✅ Strong compliance	✅ Strong compliance	✅ Strong compliance
CoT	✅ Very effective	✅ Built-in (o1 thinks automatically)	✅ Effective
Long context (200K+)	✅ Best in class	✅ Good (128K)	✅ Best capacity (1M)
Prefilling	✅ Supported via assistant role	⚠️ Partial via system	❌ Limited support
Few-shot examples	✅ Highly effective	✅ Highly effective	✅ Effective
Code generation	✅ Excellent	✅ Excellent	✅ Good (strong on Python)

Model Selection Decision Matrix

TASK	RECOMMENDED	ALTERNATIVE
Long document analysis (>100K tokens)	Claude Sonnet / Gemini 1.5	GPT-4o with chunking
Complex multi-step reasoning	o1 / Claude Opus	Claude Sonnet with CoT
Code generation (complex)	Claude Sonnet / GPT-4o	Either works well
High-volume classification	Claude Haiku / GPT-4o-mini	Gemini Flash
Creative writing	Claude Sonnet	GPT-4o
Structured data extraction	Claude Sonnet (with XML)	GPT-4o with JSON mode

APPENDIX C

CLAUDE.md Template Library

Starter CLAUDE.md

```
# CLAUDE.md  -  Project Operating Manual
# Updated: [DATE] | v1.0

## Critical Directives
1. [Most important constraint  -  e.g., never commit secrets]
2. [Second most important]
3. [Third most important]

## Tech Stack
- Language: [language + version]
- Framework: [framework + version]
- Database: [db + version]
- Testing: [framework]

## Key Commands
[build command]   # [what it does]
[test command]    # [what it does]
[lint command]    # [what it does]

## Canonical Locations
| Category | Location |
|----------|----------|
| [type] | [path] |

## Development Patterns
- [Pattern 1]: [one sentence]
- [Pattern 2]: [one sentence]
```

Developer CLAUDE.md (Full)

```
# CLAUDE.md  -  Developer Edition

## ⚡ Critical
1. Tests MUST pass before any PR: [test command]
2. Zero linting violations: [lint command]
3. Read before modifying  -  use Reconnaissance pattern
4. Never commit: .env, credentials, API keys

## Stack
- TypeScript 5.x strict + Node 20 LTS
- PostgreSQL 16 + Prisma ORM
- Vitest + Testing Library + Supertest
- ESLint + Prettier (config: .eslintrc.json + .prettierrc)

## Commands
npm run dev           # Dev server (port 3000)
npm run test          # Full test suite
npm run test:watch    # TDD mode
npm run lint          # Zero tolerance policy
npm run type-check    # TypeScript validation
npm run migrate       # Run pending DB migrations
npm run seed          # Seed development database

## Structure
src/
├── routes/          # Express route handlers
├── services/        # Business logic
├── lib/             # Shared utilities
|   ├── errors.ts    # AppError class (use this, not raw Error)
|   └── db.ts        # Prisma client singleton
├── middleware/      # Express middleware (auth.ts, validate.ts)
└── types/           # Global TypeScript types

## Error Handling
- Use AppError(message, statusCode) for all API errors
- Never throw raw Error  -  context is lost
- Log errors via logger.error (src/lib/logger.ts) not console.error

## Testing Conventions
- Test files: alongside source (*.test.ts)
- Integration tests: tests/integration/
- Use factories (tests/factories/) not raw db inserts
- Test descriptions: "should [behavior] when [condition]"
```

Research CLAUDE.md

```
# CLAUDE.md - Research Assistant Configuration

## Research Protocol
1. State what you know vs. what you're inferring
2. Cite sources for all factual claims
3. Flag when information may be out of date
4. Distinguish: primary sources vs. secondary commentary

## Output Conventions
- Research briefs: research-sessions/YYYY-MM-DD-[topic].md
- Citation format: [Author, Year, Title, URL]
- Confidence levels: HIGH (primary sources) / MEDIUM (reputable secondary) / LOW (in
ference)

## Anti-Patterns
- Never present speculation as fact
- Never omit counterevidence
- Never cite sources you haven't verified exist
```

Content Creator CLAUDE.md

```
# CLAUDE.md - Content Creator Configuration

## Voice
Short sentences. No passive voice. Technical without jargon.
Data before claims. Specific over general.
[Add 3-5 samples of your writing below for reference]

## Content Types
- Posts: max 200 words, no intro paragraph, hook first
- Articles: 600-900 words, clear structure, one key takeaway
- Threads: max 8 tweets, each standalone

## Anti-Patterns
- No "I'm excited to share"
- No "In today's world"
```

```
- No emojis unless explicitly requested
- No hedging language ("kind of," "somewhat," "I think maybe")
```

What You Now Have

30 chapters. 8 template schemas. 7 persuasion principles. 8 development patterns. 100+ battle-tested templates. Every major technique documented, ranked by impact, and demonstrated with real examples.

This isn't a collection of tricks - it's a complete engineering discipline. Prompts as code. Agents as systems. Quality as a measurable standard, not a vibe.

Part I–III

MENTAL MODELS → TECHNIQUES → SYSTEMS

Part IV–V

PSYCHOLOGY → AGENTIC PATTERNS

Part VI–VIII

PATTERNS → ANTI-PATTERNS → APPLIED

Synthesized from Anthropic's official documentation, 434 YouTube videos, academic research, and 18 months of production AI engineering. The Prompt Engineering OS - 2026 Edition.